

SQL — The Complete Guide

DBMS Foundations • RDBMS vs NoSQL • Every SQL Topic End to End

Joins • Subqueries • Window Functions • Indexes • Transactions • Performance Tuning • Advanced SQL

Plus: Spring Batch — Real-Time Batch Processing Architecture

A practical, simple-language, end-to-end reference for interview preparation and real-world implementation

Table of Contents

1. Before SQL — What Is a DBMS and Why We Need One
2. RDBMS vs NoSQL — Full Comparison With Real-Time Scenarios
3. Introduction to SQL and Relational Databases
4. DDL, DML, DCL, TCL — The Four SQL Command Categories
5. SELECT, WHERE, ORDER BY
6. GROUP BY, HAVING, DISTINCT
7. LIMIT / FETCH FIRST / TOP
8. INNER JOIN
9. LEFT JOIN & RIGHT JOIN
10. FULL OUTER JOIN, SELF JOIN & CROSS JOIN
11. Subqueries — Single Row, Multiple Row, Correlated
12. EXISTS vs IN vs NOT EXISTS vs NOT IN
13. Aggregate Functions (COUNT, SUM, AVG, MIN, MAX)
14. String Functions
15. Date Functions
16. NULL Handling Functions
17. Window Functions — Introduction & PARTITION BY
18. ROW_NUMBER, RANK & DENSE_RANK
19. LEAD, LAG, FIRST_VALUE & LAST_VALUE
20. Set Operators — UNION, UNION ALL, INTERSECT, EXCEPT/MINUS
21. Database Objects — Tables, Views & Materialized Views
22. Database Objects — Sequences & Synonyms
23. Indexes Deep Dive
24. Constraints
25. Transactions & ACID Properties
26. COMMIT, ROLLBACK, SAVEPOINT & Transaction Lifecycle
27. Concurrency, Locking & Deadlocks
28. Isolation Levels & Read Phenomena
29. Query Optimization & Execution Plans
30. Indexing Strategy & SQL Tuning
31. Common Table Expressions (CTE) & Recursive CTE
32. Temporary Tables & the MERGE Statement
33. Stored Procedures, Functions & Triggers
34. Real-Time SQL Development in Organizations
35. Common SQL Issues & Real-Time Scenarios
36. SQL Interview Questions & Answers (Most Important Topics)
37. What Is Batch Processing and Why We Need It (Real-Time)
38. Introduction to Spring Batch
39. Spring Batch Architecture
40. Spring Batch Core Concepts — Job, Step, ItemReader, ItemProcessor, ItemWriter
41. Spring Batch — Real Implementation Example (End to End)
42. Chunk-Oriented vs Tasklet Processing
43. Scaling Spring Batch — Partitioning, Multi-Threading & Parallel Steps
44. Scheduling, Restart, Retry & Skip Logic
45. Spring Batch Real-Time Scenarios & Issues

46. Spring Batch Interview Questions & Answers

47. Final Summary & Cheat Sheet

1. Before SQL — What Is a DBMS and Why We Need One

1.1 Life Before Databases

Before database systems, applications stored data in flat files (CSV, plain text) — simple, but with no built-in way to enforce data integrity, handle concurrent access safely, or query efficiently across large volumes of data.

1.2 Real Problems Flat Files Caused

Problem	Why It Hurt
No concurrent access control	Two programs writing to the same file at once could corrupt data
No data integrity enforcement	Nothing stopped duplicate IDs, invalid references, or missing required fields
No efficient querying	Finding one record meant scanning the entire file line by line
No relationships between data	Connecting "customer" data to "orders" data required manual, error-prone file-matching logic
No transaction safety	A crash mid-write could leave a file in a half-updated, corrupted state

1.3 What a DBMS Is

A DBMS (Database Management System) is software that manages data storage, retrieval, security, concurrency, and integrity for you — instead of each application reinventing file handling, the DBMS provides a structured, reliable, and queryable way to store and access data.

1.4 Types of DBMS

Type	Data Model	Real Examples
Relational (RDBMS)	Structured tables with rows/columns, related via keys	MySQL, PostgreSQL, Oracle, SQL Server
NoSQL — Document	Flexible JSON-like documents	MongoDB, CouchDB
NoSQL — Key-Value	Simple key-to-value pairs	Redis, DynamoDB
NoSQL — Column-Family	Wide-column, optimized for huge write volumes	Cassandra, HBase
NoSQL — Graph	Nodes and relationships as first-class concepts	Neo4j

This guide focuses on **SQL** — the language used to interact with Relational Database Management Systems (RDBMS) — since it remains the backbone of most real-world Java/Spring Boot enterprise applications, with NoSQL used for specific, complementary use cases (Section 2).

2. RDBMS vs NoSQL — Full Comparison With Real-Time Scenarios

2.1 The Core Philosophical Difference

RDBMS vs NoSQL — Structure

RDBMS: Fixed schema, tables with rows/columns, strong relationships via foreign keys

NoSQL: Flexible/schemaless, documents/key-values/wide-columns, relationships often denormalized

2.2 Full Comparison Table

Aspect	RDBMS	NoSQL
Schema	Fixed, strictly defined upfront	Flexible/dynamic, can evolve per document
Consistency Model	Strong consistency (ACID)	Often eventual consistency (BASE), though some support strong consistency too
Relationships	First-class — foreign keys, joins	Usually denormalized/embedded, joins are limited or absent
Scaling	Traditionally vertical (bigger server); horizontal scaling is harder	Built for horizontal scaling (sharding across many servers) from the start
Query Language	SQL — standardized, powerful, declarative	Varies by product (MongoDB query syntax, CQL for Cassandra, etc.)
Transactions	Full ACID transactions across multiple tables/rows	Limited or single-document transactions historically (improving in modern versions)
Best fit	Complex relational data, strong consistency needs (banking, financial records)	Massive scale, flexible/evolving schemas, high write throughput (logs, IoT, catalogs)

2.3 Real-Time Scenario — RDBMS

Banking core system (Accounts, Cards, Loans): requires strict ACID transactions (a fund transfer must be fully atomic), strong relational integrity (an Account must reference a valid Customer), and regulatory-grade consistency — RDBMS (PostgreSQL/Oracle) is the clear right choice. Losing a single cent due to eventual consistency is unacceptable in banking.

2.4 Real-Time Scenario — NoSQL

Product catalog for an e-commerce platform: millions of products with wildly varying attributes (a shirt has size/color, a laptop has RAM/CPU) — a rigid RDBMS schema would need constant migrations. MongoDB's flexible document model lets

each product store exactly the attributes it needs, and the catalog needs to scale horizontally to handle huge read traffic — a great NoSQL fit.

IoT sensor data / application logs: extremely high write volume, simple structure, rarely needs complex joins — Cassandra's column-family model and horizontal write scaling handles this far better than a single RDBMS instance would.

2.5 Real Organizations Often Use Both Together (Polyglot Persistence)

Real Polyglot Persistence — Banking Example

PostgreSQL: Accounts, Cards, Loans (strict ACID)

Redis: Session/cache data (fast key-value)

Elasticsearch: Transaction search/logs (full-text search at scale)

Interview one-liner: "RDBMS and NoSQL aren't competitors to pick once for an entire company — real organizations use RDBMS for strongly relational, transactional core data, and NoSQL for specific use cases (flexible catalogs, massive write-scale logs, caching) where its trade-offs are actually a better fit."

3. Introduction to SQL and Relational Databases

3.1 What SQL Is

SQL (Structured Query Language) is the standard language for defining, querying, and managing data in a relational database — despite different RDBMS vendors having their own extensions/dialects, the core SQL syntax is remarkably consistent across MySQL, PostgreSQL, Oracle, and SQL Server.

3.2 Core Relational Concepts

Concept	Meaning
Table	A structured collection of rows, each with the same set of columns
Row (Record/Tuple)	One single entry in a table
Column (Field/Attribute)	One piece of data each row has, with a defined data type
Primary Key	A column (or set of columns) uniquely identifying each row (Section 24)
Foreign Key	A column referencing a primary key in another table, enforcing relationships (Section 24)
Schema	The overall structure — tables, columns, relationships, constraints

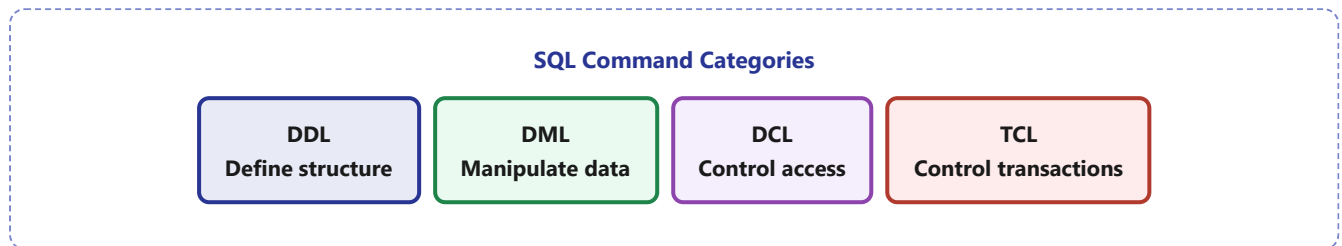
3.3 The Banking Example Schema Used Throughout This Guide

```
customer(id, name, email, city, created_date)
account(id, customer_id, balance, status, account_type, created_date)
card(id, account_id, card_number, limit_amount, status)
loan(id, customer_id, amount, interest_rate, status, approved_date)
transaction(id, account_id, amount, transaction_type, transaction_date)
```

All examples throughout this guide use this same consistent banking schema — customer, account, card, loan, transaction — so every query builds intuitively on a schema you already understand.

4. DDL, DML, DCL, TCL — The Four SQL Command Categories

4.1 The Four Categories, Visualized



4.2 DDL (Data Definition Language)

```
CREATE TABLE account (  
    id BIGSERIAL PRIMARY KEY,  
    customer_id BIGINT NOT NULL REFERENCES customer(id),  
    balance NUMERIC(15,2) NOT NULL DEFAULT 0,  
    status VARCHAR(20) NOT NULL  
);  
  
ALTER TABLE account ADD COLUMN account_type VARCHAR(20);  
ALTER TABLE account DROP COLUMN account_type;  
  
DROP TABLE account;           -- removes the table AND all its data, structure gone entirely  
TRUNCATE TABLE transaction;    -- removes ALL rows fast, keeps the table structure intact
```

Command	Purpose
CREATE	Creates a new table/view/index/other database object
ALTER	Modifies an existing object's structure
DROP	Permanently removes an object and its data
TRUNCATE	Quickly removes all rows from a table (faster than DELETE, can't be rolled back in most databases, resets identity counters)

4.3 DML (Data Manipulation Language)

```
SELECT * FROM account WHERE status = 'ACTIVE';  
INSERT INTO account (customer_id, balance, status) VALUES (55, 5000.00, 'ACTIVE');  
UPDATE account SET balance = balance - 500 WHERE id = 1001;  
DELETE FROM account WHERE id = 1001;
```

4.4 DCL (Data Control Language)

```
GRANT SELECT, INSERT ON account TO app_readonly_role;  
REVOKE INSERT ON account FROM app_readonly_role;
```

DCL controls WHO can do WHAT — real organizations use this to enforce least-privilege access, e.g., a reporting/analytics service account might only ever be granted SELECT, never INSERT/UPDATE/DELETE.

4.5 TCL (Transaction Control Language)

```
BEGIN;                                -- start a transaction
UPDATE account SET balance = balance - 500 WHERE id = 1001;
UPDATE account SET balance = balance + 500 WHERE id = 1002;
COMMIT;                                -- make the changes permanent
-- or: ROLLBACK; to undo everything since BEGIN
```

Full transaction detail (ACID, SAVEPOINT, isolation levels) is covered in depth in Sections 24-28 — this section just establishes TCL as the fourth command category alongside DDL/DML/DCL.

5. SELECT, WHERE, ORDER BY

5.1 The Basic SELECT Statement

```
SELECT id, balance, status FROM account;           -- specific columns (always prefer this over SELECT *)
SELECT * FROM account;                             -- all columns – avoid in production code, fine for ad-
hoc exploration
SELECT DISTINCT status FROM account;               -- unique values only (Section 6.3)
```

5.2 WHERE — Filtering Rows

```
SELECT * FROM account WHERE status = 'ACTIVE';
SELECT * FROM account WHERE balance > 10000;
SELECT * FROM account WHERE balance BETWEEN 1000 AND 5000;
SELECT * FROM account WHERE status IN ('ACTIVE', 'PENDING');
SELECT * FROM customer WHERE email LIKE '%@gmail.com';    -- pattern match
SELECT * FROM account WHERE closed_date IS NULL;          -- never use "= NULL", always IS NULL
SELECT * FROM account WHERE status = 'ACTIVE' AND balance > 5000;
SELECT * FROM account WHERE status = 'ACTIVE' OR status = 'PENDING';
SELECT * FROM account WHERE NOT status = 'CLOSED';
```

Operator	Meaning
=, != (or <>), <, >, <=, >=	Standard comparison operators
BETWEEN x AND y	Inclusive range check
IN (a, b, c)	Matches any value in the list
LIKE	Pattern matching — % = any characters, _ = one character
IS NULL / IS NOT NULL	Null checks — SQL's three-valued logic means = NULL never matches anything
AND / OR / NOT	Logical combinators

5.3 ORDER BY — Sorting Results

```
SELECT * FROM account ORDER BY balance DESC;           -- highest balance first
SELECT * FROM account ORDER BY status ASC, balance DESC; -- multi-column sort
SELECT * FROM customer ORDER BY city NULLS LAST;       -- control where NULLs appear in sort order
```

Real scenario: "A support dashboard listing 'accounts to review' sorts by **ORDER BY risk_score DESC, balance DESC** — highest risk, highest balance accounts always appear first, so the team's limited review time is spent where it matters most."

6. GROUP BY, HAVING, DISTINCT

6.1 GROUP BY — Aggregating Rows Into Groups

```
SELECT status, COUNT(*) AS account_count
FROM account
GROUP BY status;
-- Result: ACTIVE | 4200, CLOSED | 850, PENDING | 60

SELECT customer_id, SUM(balance) AS total_balance
FROM account
GROUP BY customer_id;
```

Every column in the SELECT list that ISN'T inside an aggregate function (COUNT, SUM, etc.) MUST appear in the GROUP BY clause — this is one of the most common real SQL errors for beginners.

6.2 HAVING — Filtering AFTER Grouping (vs WHERE Which Filters BEFORE)

```
SELECT customer_id, SUM(balance) AS total_balance
FROM account
GROUP BY customer_id
HAVING SUM(balance) > 100000;           -- can't use WHERE here - aggregates don't exist until AFTER grouping

-- Combining WHERE (filters individual rows first) and HAVING (filters the resulting groups)
SELECT customer_id, COUNT(*) AS active_accounts
FROM account
WHERE status = 'ACTIVE'
GROUP BY customer_id
HAVING COUNT(*) > 1;                   -- customers with MORE THAN ONE active account
```

Real SQL Execution Order (Not Written Order!)



Very common interview question: WHERE filters individual rows BEFORE grouping happens; HAVING filters the resulting GROUPS after aggregation — you cannot use an aggregate function like `SUM()` inside a WHERE clause because at that stage, grouping hasn't happened yet.

6.3 DISTINCT — Removing Duplicates

```
SELECT DISTINCT status FROM account;           -- unique statuses only
SELECT DISTINCT city, status FROM customer JOIN account ...; -- unique COMBINATIONS of both columns
SELECT COUNT(DISTINCT customer_id) FROM account; -- count of unique customers who have at least
one account
```

7. LIMIT / FETCH FIRST / TOP

7.1 Database-Specific Row-Limiting Syntax

Database	Syntax
PostgreSQL / MySQL	<code>SELECT * FROM account ORDER BY balance DESC LIMIT 10;</code>
SQL Server	<code>SELECT TOP 10 * FROM account ORDER BY balance DESC;</code>
Oracle / ANSI SQL Standard	<code>SELECT * FROM account ORDER BY balance DESC FETCH FIRST 10 ROWS ONLY;</code>

7.2 Combining With OFFSET for Pagination

```
-- PostgreSQL/MySQL – page 3, 20 rows per page (skip the first 40 rows)
SELECT * FROM account ORDER BY id LIMIT 20 OFFSET 40;

-- ANSI Standard equivalent
SELECT * FROM account ORDER BY id OFFSET 40 ROWS FETCH NEXT 20 ROWS ONLY;
```

Real performance gotcha: Large OFFSET values (e.g., `OFFSET 500000`) are slow — the database still has to scan and discard all 500,000 skipped rows internally. For genuinely deep pagination at scale, real systems use **keyset pagination** instead: `WHERE id > :lastSeenId ORDER BY id LIMIT 20`, which uses an index seek instead of scanning/discarding rows.

8. INNER JOIN

8.1 What INNER JOIN Does

INNER JOIN returns only rows that have a MATCH in BOTH tables — if a customer has no accounts, or an account somehow has no valid customer, neither appears in the result at all.

INNER JOIN — Only the Overlap

Customer

n INNER JOIN n

Account

Result = only customers WITH accounts, matched to their accounts

8.2 Real Query

```
SELECT c.name, a.balance, a.status
FROM customer c
INNER JOIN account a ON c.id = a.customer_id
WHERE a.status = 'ACTIVE';

-- Joining three tables – a very common real pattern
SELECT c.name, a.balance, l.amount AS loan_amount, l.status AS loan_status
FROM customer c
INNER JOIN account a ON c.id = a.customer_id
INNER JOIN loan l ON c.id = l.customer_id
WHERE l.status = 'ACTIVE';
```

Real scenario: "Our 'active customers with active accounts' report uses INNER JOIN deliberately — customers who never opened an account (perhaps abandoned signup) should NOT appear in this specific report at all, which is exactly INNER JOIN's behavior."

9. LEFT JOIN & RIGHT JOIN

9.1 LEFT JOIN — Keep Everything From the Left Table

LEFT JOIN returns ALL rows from the left (first) table, plus matching rows from the right table — if there's no match, the right table's columns come back as NULL instead of the row disappearing entirely.

LEFT JOIN — Everything on the Left, Matched Where Possible

ALL Customers (even those with ZERO accounts)

↓ LEFT JOIN Account

Matched account data, or NULL if the customer has no account

```
SELECT c.name, a.balance
FROM customer c
LEFT JOIN account a ON c.id = a.customer_id;
-- customers with NO accounts still appear, with a.balance shown as NULL

-- Real use: find customers who have NEVER opened an account
SELECT c.name
FROM customer c
LEFT JOIN account a ON c.id = a.customer_id
WHERE a.id IS NULL;    -- the "anti-join" pattern — extremely common real-world use of LEFT JOIN
```

9.2 RIGHT JOIN — The Mirror Image

```
SELECT c.name, a.balance
FROM customer c
RIGHT JOIN account a ON c.id = a.customer_id;
-- returns ALL accounts, even ones with a broken/orphaned customer_id (rare with proper foreign keys)
```

Real-world note: RIGHT JOIN is functionally identical to swapping the table order and using LEFT JOIN — most real organizations standardize on **always using LEFT JOIN** for readability/consistency, and simply never write RIGHT JOIN in practice, even though it's valid SQL.

10. FULL OUTER JOIN, SELF JOIN & CROSS JOIN

10.1 FULL OUTER JOIN

Returns ALL rows from BOTH tables — matched where possible, with NULLs on whichever side has no match. Combines the effect of LEFT JOIN and RIGHT JOIN together.

```
SELECT c.name, a.balance
FROM customer c
FULL OUTER JOIN account a ON c.id = a.customer_id;
-- shows: customers with no accounts (NULL balance), AND any orphaned accounts (NULL name)
```

MySQL does not support **FULL OUTER JOIN** directly — it's emulated using **LEFT JOIN ... UNION ... RIGHT JOIN**. PostgreSQL, SQL Server, and Oracle all support it natively.

10.2 SELF JOIN — Joining a Table to Itself

Used when rows in the SAME table have a relationship to each other — e.g., an "employee" table where each employee has a "manager_id" pointing to another row in the same table.

```
SELECT e.name AS employee_name, m.name AS manager_name
FROM employee e
LEFT JOIN employee m ON e.manager_id = m.id;
-- the SAME table, aliased twice (e and m), to represent two different "roles" of the same data

-- Banking example — find pairs of accounts belonging to the same customer
SELECT a1.id AS account_1, a2.id AS account_2, a1.customer_id
FROM account a1
JOIN account a2 ON a1.customer_id = a2.customer_id AND a1.id < a2.id;
```

10.3 CROSS JOIN — Every Combination (Cartesian Product)

Returns every possible combination of rows from both tables — if Table A has 5 rows and Table B has 3 rows, CROSS JOIN produces 15 rows. Rarely used directly for reporting, but useful for generating combinations.

```
SELECT p.product_name, m.month_name
FROM product p
CROSS JOIN month_lookup m;
-- generates one row per product PER month — useful for building a complete report template
-- that later gets LEFT JOINed against actual sales data, ensuring months with $0 sales still appear
```

Real accidental gotcha: Forgetting the **ON** condition in a JOIN (or writing **FROM table_a, table_b** with no WHERE linking them) accidentally produces a CROSS JOIN — a classic real-world bug that silently multiplies your result set size and can look like "duplicate data" without an obvious cause.

11. Subqueries — Single Row, Multiple Row, Correlated

11.1 Single Row Subquery

Returns exactly ONE row/value — safe to use with `=`, `>`, `<`, etc.

```
SELECT * FROM account
WHERE balance > (SELECT AVG(balance) FROM account);
-- accounts with an above-average balance
```

11.2 Multiple Row Subquery

Returns MULTIPLE rows — must be used with `IN`, `ANY`, `ALL`, or `EXISTS`, never bare `=` / `>`.

```
SELECT * FROM customer
WHERE id IN (SELECT customer_id FROM loan WHERE status = 'ACTIVE');
-- customers who have at least one active loan

SELECT * FROM account
WHERE balance > ALL (SELECT balance FROM account WHERE status = 'CLOSED');
-- accounts with a balance greater than EVERY closed account's balance
```

11.3 Correlated Subquery

A correlated subquery references a column from the OUTER query — it's effectively re-executed once for each row of the outer query, unlike a normal subquery which runs once independently.

```
SELECT c.name,
       (SELECT COUNT(*) FROM account a WHERE a.customer_id = c.id) AS account_count
FROM customer c;
-- the inner subquery runs ONCE PER customer row, referencing that row's specific c.id each time

-- Correlated subquery in a WHERE clause
SELECT * FROM account a
WHERE balance > (
    SELECT AVG(balance) FROM account a2 WHERE a2.customer_id = a.customer_id
);
-- accounts with an above-average balance COMPARED ONLY TO THAT SAME CUSTOMER'S OTHER accounts
```

Real performance note: Correlated subqueries can be slow on large tables since they conceptually re-run per outer row — real query tuning often rewrites a correlated subquery as a JOIN with GROUP BY, or uses a window function (Section 17) instead, which the database can usually execute far more efficiently in a single pass.

12. EXISTS vs IN vs NOT EXISTS vs NOT IN

12.1 EXISTS and IN — Functionally Similar, Different Performance Characteristics

```
-- Using IN
SELECT * FROM customer c
WHERE c.id IN (SELECT customer_id FROM account WHERE status = 'ACTIVE');

-- Using EXISTS — often preferred, especially on large tables
SELECT * FROM customer c
WHERE EXISTS (SELECT 1 FROM account a WHERE a.customer_id = c.id AND a.status = 'ACTIVE');
```

Aspect	IN	EXISTS
How it works	Builds the full subquery result list first, then checks membership	Stops checking as soon as ONE matching row is found (short-circuits)
Performance on large subquery result sets	Can be slower — must materialize the whole list	Usually faster — doesn't need the full list, just "does at least one match exist?"
NULL handling	Dangerous — see Section 12.2	Safe — unaffected by NULLs in the subquery

12.2 The NOT IN + NULL Trap (A Very Common Real Bug)

Critical gotcha: If the subquery in a **NOT IN** can return even ONE NULL value, the ENTIRE outer query returns ZERO rows — silently, with no error. This has caused real production bugs where a report mysteriously returned empty results.

```
-- DANGEROUS — if ANY account.customer_id is NULL (e.g., an orphaned/bad record), this returns NOTHING at all
SELECT * FROM customer
WHERE id NOT IN (SELECT customer_id FROM account);

-- SAFE FIX 1 — filter out NULLs explicitly in the subquery
SELECT * FROM customer
WHERE id NOT IN (SELECT customer_id FROM account WHERE customer_id IS NOT NULL);

-- SAFE FIX 2 (preferred) — use NOT EXISTS instead, which is immune to this NULL trap entirely
SELECT * FROM customer c
WHERE NOT EXISTS (SELECT 1 FROM account a WHERE a.customer_id = c.id);
```

Real-world guidance: Because of this NULL trap, many experienced SQL developers default to **NOT EXISTS** over **NOT IN** as a blanket rule, unless they're 100% certain the subquery's column can never contain NULL.

13. Aggregate Functions (COUNT, SUM, AVG, MIN, MAX)

13.1 The Five Core Aggregate Functions

```
SELECT COUNT(*) FROM account;           -- total row count, including NULLs
SELECT COUNT(closed_date) FROM account; -- counts only NON-NULL closed_date values
SELECT COUNT(DISTINCT customer_id) FROM account; -- count of unique customers
SELECT SUM(balance) FROM account WHERE status = 'ACTIVE';
SELECT AVG(balance) FROM account;
SELECT MIN(balance), MAX(balance) FROM account;
```

13.2 Aggregates Combined With GROUP BY (Recap From Section 6)

```
SELECT account_type, COUNT(*) AS total, SUM(balance) AS total_balance, AVG(balance) AS avg_balance
FROM account
GROUP BY account_type
ORDER BY total_balance DESC;
```

13.3 Real Report Query — Multiple Aggregates Together

```
SELECT
  c.city,
  COUNT(DISTINCT c.id) AS total_customers,
  COUNT(a.id) AS total_accounts,
  SUM(a.balance) AS total_deposits,
  ROUND(AVG(a.balance), 2) AS avg_balance
FROM customer c
LEFT JOIN account a ON c.id = a.customer_id
GROUP BY c.city
HAVING COUNT(a.id) > 10
ORDER BY total_deposits DESC;
```

Real scenario: "Our regional performance dashboard runs exactly this style of query nightly — aggregating deposits/customer counts per city — feeding a Grafana panel the business team checks every morning to spot underperforming branches."

14. String Functions

Function	Purpose	Example
CONCAT /	Joins strings together	<code>CONCAT(first_name, ' ', last_name)</code>
UPPER / LOWER	Case conversion	<code>UPPER(email)</code>
TRIM / LTRIM / RTRIM	Removes whitespace	<code>TRIM(name)</code>
LENGTH / LEN	String length	<code>LENGTH(card_number)</code>
SUBSTRING	Extracts part of a string	<code>SUBSTRING(card_number, 13, 4)</code> — last 4 digits
REPLACE	Replaces a substring	<code>REPLACE(phone, '-', '')</code>
LPAD / RPAD	Pads a string to a fixed length	<code>LPAD(account_id::text, 10, '0')</code>
POSITION / INSTR	Finds a substring's position	<code>POSITION('@' IN email)</code>

14.1 Real Query — Masking Sensitive Data

```
SELECT
    CONCAT('****-****-****-', SUBSTRING(card_number, 13, 4)) AS masked_card_number
FROM card;
-- real security practice – never display full card numbers in reports/dashboards
```

14.2 Real Query — Data Cleanup

```
UPDATE customer SET email = LOWER(TRIM(email)) WHERE email != LOWER(TRIM(email));
-- normalizes inconsistent email casing/whitespace before enforcing a unique constraint (Section 24)
```

15. Date Functions

Function	Purpose	Example
NOW() / CURRENT_TIMESTAMP	Current date and time	<code>SELECT NOW();</code>
CURRENT_DATE	Current date only	<code>SELECT CURRENT_DATE;</code>
DATE_PART / EXTRACT	Extracts a specific part (year, month, day)	<code>EXTRACT(YEAR FROM created_date)</code>
DATEADD / + INTERVAL	Adds/subtracts time	<code>created_date + INTERVAL '30 days'</code>
DATEDIFF / (date1 - date2)	Difference between two dates	<code>closed_date - opened_date</code>
TO_CHAR / FORMAT	Formats a date as a string	<code>TO_CHAR(created_date, 'YYYY-MM-DD')</code>
TO_DATE / CAST	Parses a string into a date	<code>TO_DATE('2026-01-15', 'YYYY-MM-DD')</code>

15.1 Real Query — Monthly Report Grouping

```
SELECT
  EXTRACT(YEAR FROM transaction_date) AS txn_year,
  EXTRACT(MONTH FROM transaction_date) AS txn_month,
  COUNT(*) AS txn_count,
  SUM(amount) AS total_amount
FROM transaction
GROUP BY EXTRACT(YEAR FROM transaction_date), EXTRACT(MONTH FROM transaction_date)
ORDER BY txn_year, txn_month;
```

15.2 Real Query — Finding Overdue Loans

```
SELECT * FROM loan
WHERE status = 'ACTIVE'
  AND next_payment_date < CURRENT_DATE;
-- loans past their due date – a real, frequently scheduled batch report (connects to Spring Batch, Section 37+)

SELECT * FROM account
WHERE created_date >= CURRENT_DATE - INTERVAL '90 days';
-- accounts opened in the last 90 days
```

16. NULL Handling Functions

16.1 Why NULL Handling Functions Matter

NULL represents "unknown/missing," and arithmetic/comparisons involving NULL don't behave like normal values — `NULL + 5` is NULL, not 5; `NULL = NULL` evaluates to unknown (not true). Dedicated functions let you handle these cases explicitly and safely.

16.2 COALESCE — The Most Important One

```
SELECT COALESCE(nickname, first_name, 'Unknown') AS display_name
FROM customer;
-- returns the FIRST non-null value in the list – extremely common real pattern for fallback defaults

SELECT SUM(COALESCE(discount_amount, 0)) FROM orders;
-- treats NULL discount as 0 instead of breaking the SUM
```

16.3 NULLIF — The Inverse Idea

```
SELECT amount / NULLIF(quantity, 0) AS unit_price FROM orders;
-- NULLIF returns NULL if quantity=0, converting a potential division-by-zero ERROR into a safe NULL result
```

16.4 IS NULL / IS NOT NULL vs = NULL (Recap From Section 5.2)

```
SELECT * FROM account WHERE closed_date IS NULL;      -- correct
SELECT * FROM account WHERE closed_date = NULL;      -- WRONG – always returns zero rows, silently
```

16.5 Database-Specific Equivalents

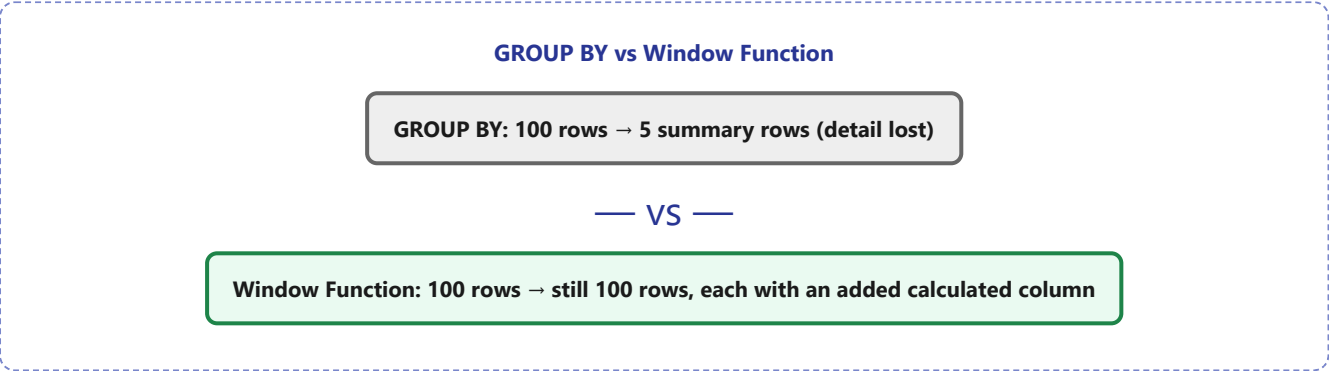
Function	Database	Behavior
ISNULL(x, default)	SQL Server	Similar to COALESCE but only takes 2 arguments
NVL(x, default)	Oracle	Oracle's equivalent of COALESCE for 2 arguments
COALESCE(x, y, z, ...)	ANSI Standard — works everywhere	Preferred for portability, accepts any number of arguments

Real scenario: "Our customer display name logic uses `COALESCE(preferred_name, first_name, 'Customer')` — this single expression gracefully handles three real data states: a customer who set a preferred name, one who didn't, and defensively falls back to a generic label if even first_name is somehow missing."

17. Window Functions — Introduction & PARTITION BY

17.1 What Makes Window Functions Different From GROUP BY

GROUP BY collapses multiple rows into ONE summary row per group, losing the individual row details. Window functions calculate an aggregate/ranking value **per row**, while still keeping every individual row in the result — the "window" is the set of related rows the calculation considers for each row.



17.2 The OVER Clause and PARTITION BY

```
SELECT
  customer_id,
  account_id,
  balance,
  SUM(balance) OVER (PARTITION BY customer_id) AS customer_total_balance
FROM account;
-- each row still shows its own account's balance, PLUS that customer's TOTAL across all their accounts
-- PARTITION BY resets the calculation separately for each customer_id group — like a "mini GROUP BY" per row
```

Result Row	customer_id	balance	customer_total_balance
1	55	2000	7000
2	55	5000	7000
3	88	3000	3000

Real scenario: "A customer dashboard shows each individual account's balance next to that customer's total combined balance across all accounts — exactly what **PARTITION BY customer_id** provides in a single query, without needing a separate subquery join."

17.3 Window Function Syntax Structure

```
function_name() OVER (  
    PARTITION BY column1      -- optional: groups rows for the calculation (like GROUP BY, but keeps rows)  
    ORDER BY column2         -- optional: defines the order within each partition (needed for  
ranking/LEAD/LAG)  
)
```

18. ROW_NUMBER, RANK & DENSE_RANK

18.1 The Three Ranking Functions Compared

```
SELECT
  customer_id,
  balance,
  ROW_NUMBER() OVER (ORDER BY balance DESC) AS row_num,
  RANK() OVER (ORDER BY balance DESC) AS rank_num,
  DENSE_RANK() OVER (ORDER BY balance DESC) AS dense_rank_num
FROM account;
```

balance	ROW_NUMBER	RANK	DENSE_RANK
5000	1	1	1
4000	2	2	2
4000 (tie)	3	2	2
3000	4	4	3

ROW_NUMBER: always unique, sequential, no gaps, no ties (arbitrary tiebreak order for equal values). **RANK:** ties share the same rank, but the NEXT rank SKIPS numbers (2, 2, then jumps to 4). **DENSE_RANK:** ties share the same rank, but the NEXT rank does NOT skip (2, 2, then 3).

18.2 Real Use Case — Top N Per Group (Extremely Common Real Pattern)

```
SELECT * FROM (
  SELECT
    customer_id,
    account_id,
    balance,
    ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY balance DESC) AS rn
  FROM account
) ranked
WHERE rn = 1;
-- returns each customer's SINGLE highest-balance account – "top 1 per group" is one of the most common real
SQL patterns
```

18.3 Real Use Case — Deduplication

```
DELETE FROM customer
WHERE id IN (
  SELECT id FROM (
    SELECT id, ROW_NUMBER() OVER (PARTITION BY email ORDER BY created_date) AS rn
    FROM customer
  ) t
  WHERE rn > 1
);
-- keeps the EARLIEST record per duplicate email, deletes the rest — a real data-cleanup pattern
```

Interview tip: "ROW_NUMBER + PARTITION BY" is one of the single most useful, most frequently asked SQL patterns in real interviews — it solves "top N per group," deduplication, and pagination-within-groups all with the same core technique.

19. LEAD, LAG, FIRST_VALUE & LAST_VALUE

19.1 LAG — Looking at the PREVIOUS Row

```
SELECT
    account_id,
    transaction_date,
    amount,
    LAG(amount) OVER (PARTITION BY account_id ORDER BY transaction_date) AS previous_amount,
    amount - LAG(amount) OVER (PARTITION BY account_id ORDER BY transaction_date) AS change_from_previous
FROM transaction;
-- compares each transaction to the PREVIOUS one for the SAME account, in date order
```

19.2 LEAD — Looking at the NEXT Row

```
SELECT
    account_id,
    transaction_date,
    amount,
    LEAD(transaction_date) OVER (PARTITION BY account_id ORDER BY transaction_date) AS next_transaction_date,
    LEAD(transaction_date) OVER (PARTITION BY account_id ORDER BY transaction_date) - transaction_date AS
days_until_next
FROM transaction;
-- calculates the gap in days until this account's NEXT transaction — great for detecting dormant accounts
```

19.3 FIRST_VALUE and LAST_VALUE

```
SELECT
    account_id,
    transaction_date,
    amount,
    FIRST_VALUE(amount) OVER (PARTITION BY account_id ORDER BY transaction_date) AS first_transaction_amount,
    LAST_VALUE(amount) OVER (
        PARTITION BY account_id ORDER BY transaction_date
        RANGE BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
    ) AS last_transaction_amount
FROM transaction;
```

Real gotcha with LAST_VALUE: By default, the "window frame" only extends up to the CURRENT row, so **LAST_VALUE** without an explicit frame clause often just returns the current row's own value, not the actual last value in the whole partition. You must explicitly specify **RANGE BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING** to get the true last value across the entire partition.

19.4 Real Scenario — Detecting a Customer's First Deposit vs Later Activity

```
SELECT customer_id, transaction_date, amount,  
       CASE WHEN transaction_date = FIRST_VALUE(transaction_date) OVER (  
         PARTITION BY customer_id ORDER BY transaction_date  
       ) THEN 'FIRST_DEPOSIT' ELSE 'SUBSEQUENT' END AS deposit_type  
FROM transaction  
WHERE transaction_type = 'DEPOSIT';
```

Real scenario: "Our fraud detection query uses `LAG(amount)` partitioned by account to flag any transaction that's more than 10x the previous transaction's amount for that same account — a simple but effective real-time anomaly signal built entirely with a window function, no separate application code needed."

20. Set Operators — UNION, UNION ALL, INTERSECT, EXCEPT/MINUS

20.1 UNION vs UNION ALL

```
SELECT customer_id FROM account WHERE status = 'ACTIVE'
UNION
SELECT customer_id FROM loan WHERE status = 'ACTIVE';
-- UNION removes duplicate customer_ids appearing in both – an implicit DISTINCT, which has a performance cost

SELECT customer_id FROM account WHERE status = 'ACTIVE'
UNION ALL
SELECT customer_id FROM loan WHERE status = 'ACTIVE';
-- UNION ALL keeps ALL rows including duplicates – faster, since no de-duplication work is needed
```

Real performance rule: Always use **UNION ALL** instead of **UNION** unless you specifically need duplicates removed — the implicit de-duplication in plain UNION requires an internal sort/hash operation that can be genuinely expensive on large result sets.

20.2 INTERSECT — Rows in BOTH Result Sets

```
SELECT customer_id FROM account WHERE status = 'ACTIVE'
INTERSECT
SELECT customer_id FROM loan WHERE status = 'ACTIVE';
-- customers who have BOTH an active account AND an active loan
```

20.3 EXCEPT (PostgreSQL/SQL Server) / MINUS (Oracle) — Rows in the First, NOT in the Second

```
-- PostgreSQL / SQL Server
SELECT customer_id FROM customer
EXCEPT
SELECT customer_id FROM account;
-- customers with NO account at all – an alternative to the LEFT JOIN ... WHERE IS NULL anti-join pattern (Section 9.1)

-- Oracle equivalent
SELECT customer_id FROM customer
MINUS
SELECT customer_id FROM account;
```

20.4 Set Operator Rules (Important)

- Both queries must return the **same number of columns**, with compatible data types in the same order.
- Column names in the final result come from the **first** query's column names/aliases.
- Only ONE **ORDER BY** is allowed, at the very end of the entire combined statement.

21. Database Objects — Tables, Views & Materialized Views

21.1 Views — A Saved, Reusable Query

A View is a virtual table defined by a stored SELECT query — it doesn't store data itself, it just re-runs its underlying query every time it's queried, always reflecting the current live data.

```
CREATE VIEW active_customer_summary AS
SELECT c.id, c.name, COUNT(a.id) AS account_count, SUM(a.balance) AS total_balance
FROM customer c
JOIN account a ON c.id = a.customer_id
WHERE a.status = 'ACTIVE'
GROUP BY c.id, c.name;

SELECT * FROM active_customer_summary WHERE total_balance > 50000;
-- queries the view just like a normal table – but it's really running the underlying JOIN/GROUP BY every time
```

Benefit	Why It Helps
Simplifies complex queries	Hides a complicated JOIN/aggregation behind a simple, reusable name
Security/access control	Can expose only specific columns/rows to certain users, hiding sensitive underlying columns
Consistency	Business logic for "what counts as an active customer" lives in ONE place, not repeated across many queries

21.2 Materialized Views — Views That Actually Store Data

Unlike a regular View, a Materialized View physically STORES the query's result set on disk — much faster to query (no re-computation each time), but the data can become stale until it's explicitly refreshed.

```
CREATE MATERIALIZED VIEW monthly_deposit_summary AS
SELECT EXTRACT(YEAR FROM transaction_date) AS yr, EXTRACT(MONTH FROM transaction_date) AS mo,
      SUM(amount) AS total_deposits
FROM transaction
WHERE transaction_type = 'DEPOSIT'
GROUP BY EXTRACT(YEAR FROM transaction_date), EXTRACT(MONTH FROM transaction_date);

REFRESH MATERIALIZED VIEW monthly_deposit_summary; -- must be run periodically (often via a scheduled job) to
update the stored data
```

Aspect	View	Materialized View
Data freshness	Always live/current (re-runs the query every time)	Can be stale until manually/scheduled refreshed
Query speed	As slow as the underlying query itself	Very fast — pre-computed and stored
Best fit	Simplifying/securing access to live data	Expensive aggregations/reports that don't need up-to-the-second freshness (e.g., a nightly dashboard)

Real scenario: "Our executive dashboard's 'monthly deposits by branch' report used to take 8 seconds to load, running a heavy aggregation query live on every page view. Converting it to a Materialized View refreshed once nightly via a scheduled job (Spring Batch, Section 37+) dropped load time to under 50ms — completely acceptable since the business only needs yesterday's numbers, not up-to-the-second accuracy, for this specific report."

22. Database Objects — Sequences & Synonyms

22.1 Sequences — Generating Unique Numbers

A Sequence is a database object that generates a guaranteed unique, typically incrementing number — commonly used to generate primary key values, especially in databases like Oracle/PostgreSQL where auto-increment isn't a plain column attribute like MySQL's AUTO_INCREMENT.

```
CREATE SEQUENCE account_id_seq START WITH 1000 INCREMENT BY 1;

SELECT nextval('account_id_seq');  -- PostgreSQL syntax - returns the next unique value, e.g., 1000, then
1001, etc.

CREATE TABLE account (
    id BIGINT PRIMARY KEY DEFAULT nextval('account_id_seq'),
    balance NUMERIC(15,2)
);
```

This connects directly back to `GenerationType.SEQUENCE` in the JDBC/Hibernate/JPA guide (Section 9.2) — Hibernate can be configured to pre-allocate a batch of sequence values at once, enabling efficient JDBC batch inserts, unlike `GenerationType.IDENTITY` which requires a round trip per row.

22.2 Synonyms — Alternate Names for Database Objects

A Synonym is an alias for another database object (a table, view, or even an object in a different schema/database) — lets you reference a long or remote object name with a short, simple, stable name, and can hide the real underlying location if it changes later.

```
CREATE SYNONYM active_accounts FOR reporting_schema.vw_active_account_summary;

SELECT * FROM active_accounts;  -- simpler than typing the fully-qualified schema.object name every time
```

Synonyms are most commonly used in Oracle-heavy enterprise environments to abstract cross-schema or cross-database references, letting the underlying physical location change without breaking every query that references it.

23. Indexes Deep Dive

23.1 What an Index Is — The Book Index Analogy

A database index is exactly like a book's index page — instead of reading every single page (a "full table scan") to find a topic, you look it up in the index and jump directly to the right page. An index is a separate, sorted data structure pointing to the actual row locations, dramatically speeding up lookups at the cost of extra storage and slightly slower writes.

23.2 Clustered vs Non-Clustered Index

	Clustered Index	Non-Clustered Index
Physical data order	The table's ACTUAL data rows are physically stored in this index's order	A separate structure with pointers back to the actual row locations
How many per table	Only ONE per table (data can only be physically sorted one way)	Many allowed per table
Typical default	Usually automatically created on the Primary Key	Created explicitly for other frequently-queried columns
Real analogy	A phone book — physically sorted by last name	An index at the back of a textbook — separate from the actual page content, just pointers

23.3 Composite Index (Multi-Column)

```
CREATE INDEX idx_account_customer_status ON account(customer_id, status);  
-- helps queries filtering by customer_id ALONE, or by customer_id AND status TOGETHER  
-- does NOT help a query filtering by status ALONE — column order in a composite index matters enormously!
```

Critical real-world rule — the "leftmost prefix" rule: A composite index on `(customer_id, status)` can be used for queries filtering on `customer_id` alone, or `customer_id + status` together — but CANNOT be used to efficiently filter on `status` alone. Always order composite index columns starting with the ones most commonly used alone or as the primary filter.

23.4 Unique Index

```
CREATE UNIQUE INDEX idx_customer_email ON customer(email);  
-- enforces no two customers can share the same email, AND speeds up lookups by email — two benefits in one
```

23.5 Covering Index

A Covering Index includes ALL the columns a query needs (in the SELECT, WHERE, and ORDER BY) directly within the index itself — the database can answer the query entirely from the index, without ever touching the actual table data ("index-only scan"), which is extremely fast.

```
-- Query needing customer_id, status, and balance
SELECT customer_id, status, balance FROM account WHERE customer_id = 55;

-- A covering index includes ALL needed columns – PostgreSQL's INCLUDE clause adds non-key columns
CREATE INDEX idx_account_covering ON account(customer_id) INCLUDE (status, balance);
```

23.6 Index Scan vs Index Seek (vs Full Table Scan)

Access Method	What Happens	Real Trigger
Full Table Scan	Reads EVERY row in the table, checking each one	No usable index exists for the query's filter condition
Index Scan	Reads through the ENTIRE index structure, still checking many entries	The index exists but isn't very selective, or a large portion of the table matches
Index Seek	Jumps DIRECTLY to the matching entries using the index's sorted structure — the fastest option	A highly selective, well-designed index exists exactly matching the query's filter

Interview tip: "Index Seek" is what you WANT to see in an execution plan for a selective query — "Full Table Scan" on a large table is usually a red flag prompting you to check whether an appropriate index exists (Section 28-29 cover reading execution plans in depth).

23.7 The Real Cost of Indexes (Not Free!)

Every index speeds up reads but slows down writes (INSERT/UPDATE/DELETE must also update every index on the table) and consumes additional disk space. Real database design deliberately balances the number of indexes against actual query patterns — indexing every single column "just in case" is a common real-world anti-pattern that degrades write performance for little read benefit.

24. Constraints

24.1 The Full Constraint Reference

Constraint	Purpose	Real Example
PRIMARY KEY	Uniquely identifies each row; implies NOT NULL + UNIQUE automatically	<code>id BIGINT PRIMARY KEY</code>
FOREIGN KEY	Enforces that a column's value must exist in another table's primary key	<code>customer_id BIGINT REFERENCES customer(id)</code>
UNIQUE	Ensures no two rows share the same value in this column	<code>email VARCHAR(100) UNIQUE</code>
CHECK	Enforces a custom business rule/condition on the column's value	<code>CHECK (balance >= 0)</code>
DEFAULT	Provides an automatic value when none is explicitly given on insert	<code>status VARCHAR(20) DEFAULT 'ACTIVE'</code>
NOT NULL	Requires the column to always have a value	<code>customer_id BIGINT NOT NULL</code>

24.2 Real Full Table Definition Using All Constraints Together

```
CREATE TABLE account (  
  id BIGINT PRIMARY KEY DEFAULT nextval('account_id_seq'),  
  customer_id BIGINT NOT NULL REFERENCES customer(id),  
  balance NUMERIC(15,2) NOT NULL DEFAULT 0 CHECK (balance >= 0),  
  account_type VARCHAR(20) NOT NULL,  
  status VARCHAR(20) NOT NULL DEFAULT 'ACTIVE',  
  account_number VARCHAR(20) UNIQUE NOT NULL,  
  created_date TIMESTAMP NOT NULL DEFAULT NOW()  
);
```

24.3 Foreign Key Referential Actions

```
CREATE TABLE account (  
  ...  
  customer_id BIGINT REFERENCES customer(id)  
    ON DELETE RESTRICT      -- prevents deleting a customer who still has accounts (SAFEST for financial  
data)  
    ON UPDATE CASCADE      -- if a customer's id somehow changes, automatically update all referencing  
accounts  
);
```

Action	Behavior
RESTRICT / NO ACTION	Blocks the delete/update if any referencing rows exist — the safest default for critical data
CASCADE	Automatically deletes/updates referencing rows too — dangerous for financial/audit-critical relationships (recap of the Cascade danger from the JPA guide, Section 11.3)

SET NULL	Sets the foreign key column to NULL on the referencing rows instead of deleting them
----------	--

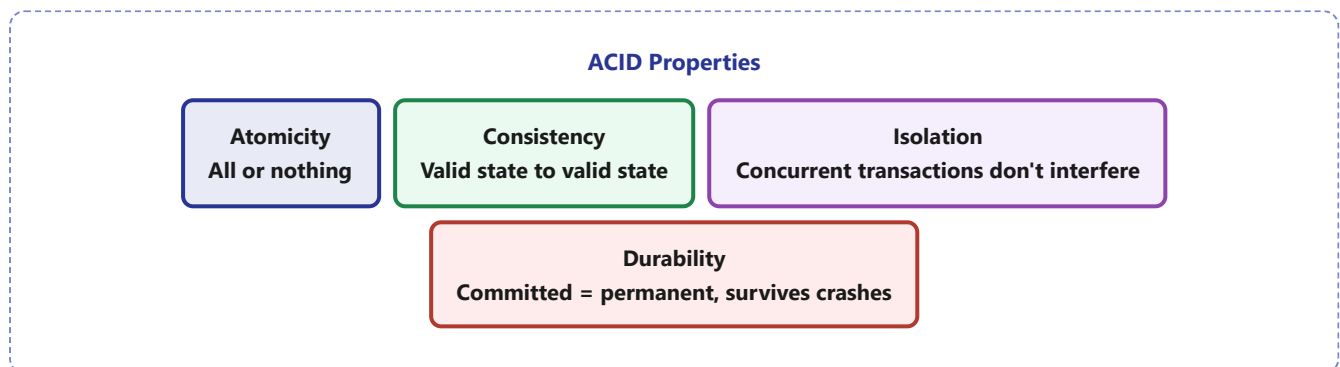
Real-world guidance: For financially/legally significant relationships (like Customer → Account), `ON DELETE RESTRICT` is almost always the correct choice — you want deleting a Customer with existing Accounts to fail loudly and force a deliberate, reviewed process, not silently cascade-delete critical financial records.

25. Transactions & ACID Properties

25.1 What a Transaction Is

A transaction is a group of one or more SQL statements that must execute as a single, indivisible unit — either ALL of them succeed and are saved permanently, or NONE of them are, even if something fails halfway through.

25.2 ACID — The Four Guarantees

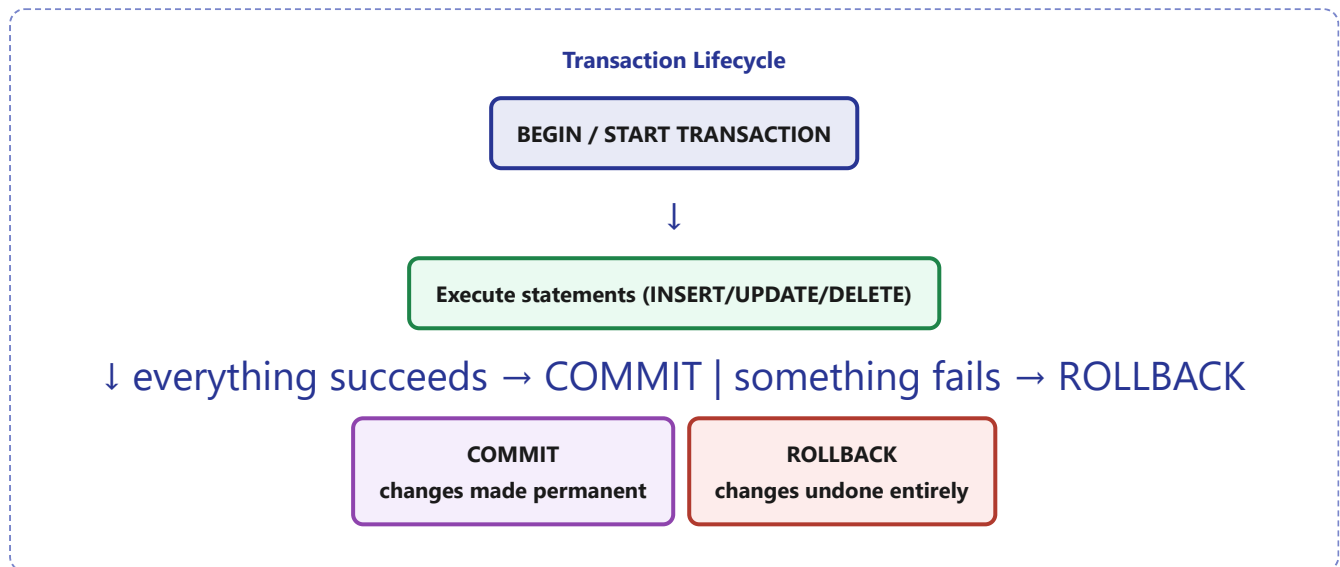


Property	Meaning	Real Banking Example
Atomicity	A transaction either fully completes or fully fails — no partial results	A fund transfer's debit AND credit both happen, or NEITHER happens — never just one
Consistency	A transaction moves the database from one valid state to another, never violating constraints	An account balance can never go negative if a CHECK constraint enforces it, even mid-transaction
Isolation	Concurrent transactions don't see each other's uncommitted intermediate changes (level configurable — Section 28)	Two simultaneous transfers on the same account don't corrupt each other's calculations
Durability	Once committed, data survives even a server crash immediately afterward	A confirmed payment is never "lost" due to a server restarting a millisecond after commit

Interview one-liner: "ACID is the set of guarantees a relational database transaction provides — Atomicity (all-or-nothing), Consistency (valid state to valid state), Isolation (concurrent transactions don't interfere), and Durability (committed data survives crashes) — together making transactions safe to build financial/critical systems on."

26. COMMIT, ROLLBACK, SAVEPOINT & Transaction Lifecycle

26.1 The Full Transaction Lifecycle



26.2 Real Transaction Example

```
BEGIN;

UPDATE account SET balance = balance - 500 WHERE id = 1001;
UPDATE account SET balance = balance + 500 WHERE id = 1002;

-- if both updates succeeded without error:
COMMIT;

-- if anything went wrong (e.g., account 1001's balance would go negative, violating a CHECK constraint):
ROLLBACK; -- undoes BOTH updates completely, as if neither ever happened
```

26.3 SAVEPOINT — Partial Rollback Within a Transaction

```
BEGIN;

UPDATE account SET balance = balance - 500 WHERE id = 1001;
SAVEPOINT after_debit;

UPDATE account SET balance = balance + 500 WHERE id = 1002;

-- if something specifically about the SECOND update needs undoing, without losing the first:
ROLLBACK TO SAVEPOINT after_debit; -- undoes only the credit, keeps the debit intact

-- then continue the transaction, perhaps crediting a different account instead
UPDATE account SET balance = balance + 500 WHERE id = 1003;
COMMIT;
```

Real scenario: "Our batch loan disbursement process uses a `SAVEPOINT` after validating and reserving funds, but before actually crediting the customer's account — if the credit step fails due to a downstream validation issue, we roll back to that savepoint and retry the credit with corrected data, without having to redo the entire fund-reservation step from scratch."

27. Concurrency, Locking & Deadlocks

27.1 Why Locking Is Necessary

When multiple transactions try to read/write the same data at the same time, the database uses locks to prevent conflicting operations from corrupting each other — e.g., two simultaneous transfers debiting the same account must not both read the same starting balance and both subtract from it independently.

27.2 Types of Locks

Lock Type	Behavior
Shared Lock (Read Lock)	Multiple transactions can hold it simultaneously — allows concurrent reads, blocks writes
Exclusive Lock (Write Lock)	Only ONE transaction can hold it — blocks both reads and writes from others until released
Row-Level Lock	Locks only the specific row(s) being modified — the common, more granular default in modern RDBMS
Table-Level Lock	Locks the ENTIRE table — much more restrictive, used for certain bulk/DDL operations

27.3 What a Deadlock Is

A deadlock happens when Transaction A holds a lock Transaction B needs, WHILE Transaction B holds a lock Transaction A needs — neither can proceed, each waiting on the other forever, until the database detects this cycle and forcibly kills one of them (rolling it back) to break the deadlock.

Deadlock Scenario

Transaction A: locks Account 1001, wants to lock Account 1002

Transaction B: locks Account 1002, wants to lock Account 1001

↓ neither can proceed — circular wait

Database detects the deadlock, kills one transaction (rollback), the other proceeds

27.4 Real Example Causing a Deadlock

```
-- Transaction A
BEGIN;
UPDATE account SET balance = balance - 100 WHERE id = 1001; -- locks row 1001
UPDATE account SET balance = balance + 100 WHERE id = 1002; -- waits for row 1002's lock...

-- Transaction B (running at nearly the same time)
BEGIN;
UPDATE account SET balance = balance - 50 WHERE id = 1002; -- locks row 1002
UPDATE account SET balance = balance + 50 WHERE id = 1001; -- waits for row 1001's lock... DEADLOCK!
```

27.5 The Real Fix — Consistent Lock Ordering

The standard real-world fix is to ALWAYS acquire locks in the same, consistent order across every transaction (e.g., always lock the lower account ID first) — this eliminates the circular wait condition that causes deadlocks entirely.

```
-- BOTH transactions now update in the SAME order (lower ID first) — no deadlock possible
UPDATE account SET balance = balance - :amount WHERE id = LEAST(:id1, :id2);
UPDATE account SET balance = balance + :amount WHERE id = GREATEST(:id1, :id2);
```

Real scenario: "Our fund transfer service used to occasionally throw deadlock exceptions under high concurrent load when two transfers happened to involve the same two accounts in opposite order. Sorting account IDs before acquiring locks (always updating the lower ID first) eliminated the deadlocks entirely — a simple, well-known real-world fix."

28. Isolation Levels & Read Phenomena

28.1 The Three Core Read Phenomena

- **Dirty Read:** Transaction A reads data that Transaction B has changed but NOT yet committed — if B rolls back, A read data that never really existed.
- **Non-Repeatable Read:** Transaction A reads the same row twice within one transaction, but gets DIFFERENT values because Transaction B updated and committed that row in between.
- **Phantom Read:** Transaction A runs the same query twice, but gets a DIFFERENT NUMBER OF ROWS because Transaction B inserted/deleted a matching row in between.

28.2 The Four Standard Isolation Levels

Isolation Level	Dirty Read	Non-Repeatable Read	Phantom Read	Performance
READ UNCOMMITTED	Possible	Possible	Possible	Fastest, least safe
READ COMMITTED (default in PostgreSQL, SQL Server, Oracle)	Prevented	Possible	Possible	Good balance, most common default
REPEATABLE READ (default in MySQL/InnoDB)	Prevented	Prevented	Possible (mostly, DB-dependent)	Slower, more locking
SERIALIZABLE	Prevented	Prevented	Prevented	Slowest, most locking/retries, but fully safe

28.3 Real Code — Setting Isolation Level Explicitly

```
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
BEGIN;
-- used sparingly, for critical operations where correctness matters more than throughput
SELECT SUM(balance) FROM account WHERE customer_id = 55;
-- ... perform reconciliation logic based on this sum ...
COMMIT;
```

28.4 Real Scenario — Choosing the Right Isolation Level

Real scenario: "Our standard account transfer operations run at the database's default READ COMMITTED isolation for good throughput under normal load. Our monthly interest calculation batch job — which reads and aggregates across the ENTIRE accounts table and cannot tolerate phantom reads (new accounts appearing mid-calculation) — explicitly runs at SERIALIZABLE isolation, accepting the performance cost for guaranteed correctness during that one critical, infrequent process."

This directly parallels the same isolation-level discussion in the Spring & Spring Boot guide (Section 44) and the JDBC/Hibernate/JPA guide (Section 17) — the database concept is identical; Spring's `@Transactional(isolation = ...)` attribute simply sets this same database-level isolation setting from application code.

29. Query Optimization & Execution Plans

29.1 What an Execution Plan Is

An Execution Plan (or Query Plan) shows EXACTLY how the database intends to execute a query — which indexes it will use (or not use), what order it joins tables in, and an estimated "cost" for each step. Reading execution plans is the single most important real-world skill for diagnosing slow queries.

29.2 Real EXPLAIN / EXPLAIN ANALYZE Usage

```
-- PostgreSQL
EXPLAIN ANALYZE
SELECT * FROM account WHERE customer_id = 55 AND status = 'ACTIVE';

-- Real simplified output to look for:
Seq Scan on account (cost=0.00..1850.00 rows=42 width=64) (actual time=12.500..45.200 rows=3 loops=1)
  Filter: (customer_id = 55 AND status = 'ACTIVE')
  Rows Removed by Filter: 49997
```

What this output reveals: `Seq Scan` means a FULL TABLE SCAN happened — the database read all ~50,000 rows and discarded 49,997 of them, just to find 3 matching rows. This is a huge red flag pointing directly at a missing index on `(customer_id, status)`.

29.3 The Same Query, After Adding the Right Index

```
CREATE INDEX idx_account_customer_status ON account(customer_id, status);

EXPLAIN ANALYZE
SELECT * FROM account WHERE customer_id = 55 AND status = 'ACTIVE';

Index Scan using idx_account_customer_status on account
 (cost=0.29..8.31 rows=3 width=64) (actual time=0.020..0.025 rows=3 loops=1)
-- cost dropped from 1850 to 8.31 – roughly 200x cheaper, and actual time dropped from 45ms to 0.025ms
```

29.4 Key Execution Plan Terms to Recognize

Term	Meaning
Seq Scan / Full Table Scan	Reads every row — usually bad on large tables unless intentionally returning most/all rows
Index Scan / Index Seek	Uses an index to find matching rows efficiently — usually good
Nested Loop Join	For each row in one table, scans the other — efficient for small result sets, bad for large ones
Hash Join	Builds an in-memory hash table from one side, then probes it — efficient for larger joins

Sort	An explicit sort operation — can be expensive on large result sets, sometimes avoidable with a matching index
Cost	The optimizer's ESTIMATED relative expense of a step — not a real time unit, but useful for comparing plans
Actual Time / Actual Rows	What genuinely happened when EXPLAIN ANALYZE really executed the query (vs just estimating)

Interview tip: `EXPLAIN` alone shows the database's ESTIMATED plan without running the query; `EXPLAIN ANALYZE` actually EXECUTES the query and shows real timing/row-count data alongside the estimates — a big real-world gap between estimated and actual rows often signals outdated table statistics needing a refresh (`ANALYZE table_name;`).

30. Indexing Strategy & SQL Tuning

30.1 A Real, Practical Indexing Strategy Checklist

- Index columns used frequently in `WHERE`, `JOIN ON`, and `ORDER BY` clauses.
- For composite indexes, put the most selective/most commonly-filtered-alone column FIRST (leftmost prefix rule, Section 23.3).
- Don't over-index — every index slows down INSERT/UPDATE/DELETE and consumes storage; only add indexes that serve real, measured query patterns.
- Periodically review and drop unused indexes (most databases can report index usage statistics).
- Keep table statistics up to date (`ANALYZE` /auto-vacuum) so the query planner makes good decisions.

30.2 Common SQL Anti-Patterns That Prevent Index Usage

```
-- BAD — a function on the indexed column prevents the index from being used at all
SELECT * FROM account WHERE UPPER(status) = 'ACTIVE';

-- GOOD — store data pre-normalized (e.g., always uppercase), or use a functional index
CREATE INDEX idx_account_status_upper ON account(UPPER(status));

-- BAD — a leading wildcard prevents most index types from helping
SELECT * FROM customer WHERE name LIKE '%smith';

-- BETTER — a trailing wildcard CAN use a standard index
SELECT * FROM customer WHERE name LIKE 'smith%';

-- BAD — implicit type conversion on the indexed column can silently disable index usage
SELECT * FROM account WHERE account_number = 12345; -- if account_number is VARCHAR, this forces a conversion

-- GOOD — match the data type exactly
SELECT * FROM account WHERE account_number = '12345';
```

30.3 Real SQL Tuning Workflow

1. Identify the slow query (via slow query logs, APM traces, or user complaints).
2. Run `EXPLAIN ANALYZE` to see the actual execution plan.
3. Look for Seq Scans on large tables, unexpectedly high actual row counts, or expensive Sort/Hash operations.
4. Check for missing indexes matching the query's actual filter/join/sort columns.
5. Check for anti-patterns (functions on indexed columns, leading wildcards, type mismatches) blocking index usage.
6. Add/adjust indexes, re-run `EXPLAIN ANALYZE` to confirm the plan improved.
7. Validate the fix under realistic data volume and concurrency, not just a small local dataset.

Real scenario: "A 'search accounts by customer name' feature was timing out under real production data volume — `WHERE UPPER(customer_name) LIKE '%' || UPPER(:search) || '%'` used both a function on the column AND a leading wildcard, defeating any standard index entirely. We introduced PostgreSQL's `pg_trgm` extension with a trigram GIN index specifically designed for this kind of flexible text search pattern, restoring sub-100ms search performance."

31. Common Table Expressions (CTE) & Recursive CTE

31.1 What a CTE Is

A CTE (defined with **WITH**) is a named, temporary result set that exists only for the duration of ONE query — it makes complex queries far more readable by breaking them into logical, named steps, similar to defining a local variable in code.

```
WITH active_high_value_customers AS (  
    SELECT customer_id, SUM(balance) AS total_balance  
    FROM account  
    WHERE status = 'ACTIVE'  
    GROUP BY customer_id  
    HAVING SUM(balance) > 100000  
)  
SELECT c.name, ahvc.total_balance  
FROM active_high_value_customers ahvc  
JOIN customer c ON c.id = ahvc.customer_id  
ORDER BY ahvc.total_balance DESC;
```

31.2 Multiple CTEs Chained Together

```
WITH monthly_totals AS (  
    SELECT customer_id, EXTRACT(MONTH FROM transaction_date) AS mo, SUM(amount) AS total  
    FROM transaction GROUP BY customer_id, EXTRACT(MONTH FROM transaction_date)  
)  
customer_averages AS (  
    SELECT customer_id, AVG(total) AS avg_monthly_total  
    FROM monthly_totals GROUP BY customer_id  
)  
SELECT c.name, ca.avg_monthly_total  
FROM customer_averages ca  
JOIN customer c ON c.id = ca.customer_id  
WHERE ca.avg_monthly_total > 5000;
```

31.3 Recursive CTE — Traversing Hierarchical Data

A Recursive CTE repeatedly references itself to walk through hierarchical/tree-structured data (organizational charts, category trees, a "manager chain") — something a plain non-recursive query cannot express at all.

```
WITH RECURSIVE employee_hierarchy AS (  
    -- Anchor member: the starting point (the top-level manager)  
    SELECT id, name, manager_id, 1 AS level  
    FROM employee  
    WHERE manager_id IS NULL  
  
    UNION ALL  
  
    -- Recursive member: joins back to the CTE itself, one level deeper each time  
    SELECT e.id, e.name, e.manager_id, eh.level + 1  
    FROM employee e  
    JOIN employee_hierarchy eh ON e.manager_id = eh.id  
)  
SELECT * FROM employee_hierarchy ORDER BY level, name;  
-- returns the ENTIRE reporting chain, from the top down, with a computed "level" depth for each person
```

Real scenario: "Our loan product catalog has nested categories (Personal Loans → Secured → Home Equity). A Recursive CTE lets us fetch a category and ALL its descendant sub-categories in one query, regardless of how many levels deep the hierarchy goes — without hardcoding a fixed number of JOIN levels."

32. Temporary Tables & the MERGE Statement

32.1 Temporary Tables — Real Use

A Temporary Table exists only for the duration of a session/transaction and is automatically dropped afterward — useful for breaking a very complex multi-step calculation into intermediate stages, especially in stored procedures/batch jobs.

```
CREATE TEMP TABLE temp_high_risk_accounts AS
SELECT id, customer_id, balance
FROM account
WHERE balance > 100000 AND status = 'ACTIVE';

-- use it like a normal table for the rest of this session
SELECT c.name, t.balance
FROM temp_high_risk_accounts t
JOIN customer c ON c.id = t.customer_id;

-- automatically dropped when the session ends (or explicitly: DROP TABLE temp_high_risk_accounts;)
```

32.2 The MERGE Statement — "Upsert" (Insert or Update)

MERGE lets you INSERT a row if it doesn't exist, or UPDATE it if it does — all in ONE statement, avoiding a separate "check then insert/update" round trip and the race condition risk that comes with it.

```
MERGE INTO account_balance_snapshot AS target
USING (SELECT id AS account_id, balance FROM account WHERE status = 'ACTIVE') AS source
ON target.account_id = source.account_id
WHEN MATCHED THEN
    UPDATE SET balance = source.balance, snapshot_date = CURRENT_DATE
WHEN NOT MATCHED THEN
    INSERT (account_id, balance, snapshot_date)
VALUES (source.account_id, source.balance, CURRENT_DATE);
```

PostgreSQL's equivalent (before native MERGE support in newer versions) is `INSERT ... ON CONFLICT ... DO UPDATE :`
`INSERT INTO account_balance_snapshot (account_id, balance) VALUES (:id, :balance) ON CONFLICT`
`(account_id) DO UPDATE SET balance = EXCLUDED.balance;`

Real scenario: "Our nightly balance-snapshot batch job (Spring Batch, Section 39-40) processes millions of accounts — using MERGE/UPSERT instead of a separate SELECT-then-INSERT-or-UPDATE per row eliminates both a race condition risk under concurrent writes AND an extra round trip per row, meaningfully speeding up the batch."

33. Stored Procedures, Functions & Triggers

33.1 Stored Procedures

A Stored Procedure is a named, precompiled block of SQL/procedural logic stored IN the database, callable by name — can accept parameters, contain control flow (IF/LOOP), and perform multiple statements/transactions as one unit.

```
CREATE OR REPLACE PROCEDURE transfer_funds(  
    p_from_id BIGINT, p_to_id BIGINT, p_amount NUMERIC  
)  
LANGUAGE plpgsql AS $$  
BEGIN  
    UPDATE account SET balance = balance - p_amount WHERE id = p_from_id;  
    UPDATE account SET balance = balance + p_amount WHERE id = p_to_id;  
  
    IF (SELECT balance FROM account WHERE id = p_from_id) < 0 THEN  
        RAISE EXCEPTION 'Insufficient funds for transfer';  
    END IF;  
END;  
$$;  
  
CALL transfer_funds(1001, 1002, 500);
```

33.2 Functions (User-Defined)

A Function is similar to a Stored Procedure but must RETURN a value, and can be used directly inside a SELECT statement — Procedures generally cannot.

```
CREATE OR REPLACE FUNCTION calculate_interest(p_principal NUMERIC, p_rate NUMERIC, p_years INT)  
RETURNS NUMERIC LANGUAGE plpgsql AS $$  
BEGIN  
    RETURN p_principal * POWER(1 + p_rate / 100, p_years) - p_principal;  
END;  
$$;  
  
SELECT id, calculate_interest(balance, 5.5, 1) AS projected_interest FROM account;
```

33.3 Triggers

A Trigger automatically fires a piece of logic in response to a table event (INSERT/UPDATE/DELETE) — without the application explicitly calling anything, useful for enforcing rules or auditing that must NEVER be bypassed, even by direct SQL.

```
CREATE OR REPLACE FUNCTION log_balance_change() RETURNS TRIGGER AS $$
BEGIN
    IF OLD.balance != NEW.balance THEN
        INSERT INTO balance_audit_log (account_id, old_balance, new_balance, changed_at)
        VALUES (NEW.id, OLD.balance, NEW.balance, NOW());
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_balance_audit
AFTER UPDATE ON account
FOR EACH ROW
EXECUTE FUNCTION log_balance_change();
-- fires AUTOMATICALLY on every balance change, from ANY source — application code, a manual SQL fix, anything
```

33.4 Real-World Trade-Offs — Where Should Logic Live?

Aspect	Logic in Application Code (Spring Boot)	Logic in Database (Procedures/Triggers)
Version control / code review	Natural fit — reviewed like any other code, in the same repo	Often separate deployment process, easier to lose track of/forget
Testability	Easy — standard unit/integration testing tools	Harder — requires specialized database testing tooling
Portability	Database-agnostic (mostly)	Tied to one specific database vendor's procedural language (PL/pgSQL, T-SQL, PL/SQL)
Guaranteed enforcement	Can be bypassed by anyone running raw SQL directly	Cannot be bypassed — fires no matter HOW the data was changed
Real-world guidance	Preferred default for most business logic in modern microservices	Reserved for genuinely database-level concerns: strict audit trails, critical invariants that must never be bypassed even by direct SQL access

Interview tip: Modern microservices architectures generally push business logic INTO the application layer (Spring Boot service classes) rather than the database, for testability, version control, and portability — triggers/procedures are reserved for cases where enforcement must be absolutely guaranteed regardless of how data is modified.

34. Real-Time SQL Development in Organizations

34.1 The Full Real-World SQL Development Checklist

1. **Design the schema** with real query patterns in mind — indexes aren't an afterthought, they're planned alongside the schema.
2. **Write every schema change as a versioned migration** (Flyway/Liquibase) — reviewed via pull request like any other code.
3. **Write queries defensively** — explicit column lists (never `SELECT *` in application code), parameterized queries (never string-concatenated SQL, to prevent SQL injection).
4. **Test with realistic data volume**, not just a handful of local rows — many performance issues only appear at real scale.
5. **Run EXPLAIN ANALYZE** on any new or modified query touching a large table before merging.
6. **Add automated tests** covering both the happy path and edge cases (NULLs, empty result sets, boundary values).
7. **Monitor slow query logs in production** continuously, not just during initial development.
8. **Review index usage periodically** — drop unused indexes, add new ones as query patterns evolve.

34.2 Real SQL Injection Prevention (Non-Negotiable)

```
-- NEVER build SQL by string concatenation with user input:
String sql = "SELECT * FROM account WHERE customer_id = " + userInput;    // SQL INJECTION VULNERABILITY

-- ALWAYS use parameterized queries / prepared statements:
PreparedStatement stmt = conn.prepareStatement("SELECT * FROM account WHERE customer_id = ?");
stmt.setLong(1, customerId);    // the driver handles safe escaping – injection becomes impossible

// Spring Data JPA/JdbcTemplate – parameterization is automatic and built in by default
@Query("SELECT a FROM Account a WHERE a.customerId = :customerId")
List<Account> findById(@Param("customerId") Long customerId);
```

34.3 Real Organization Query Review Standards

- No `SELECT *` in application code — always list explicit columns needed.
- No unparameterized/string-concatenated SQL — non-negotiable security rule.
- Every new list/report query reviewed for pagination — never fetch unbounded result sets.
- Every query joining more than 2 tables reviewed for its execution plan before merging.
- Every schema migration reviewed by at least one other engineer, checking for missing indexes on new foreign keys.

35. Common SQL Issues & Real-Time Scenarios

Issue	Cause	Real Solution
Report suddenly returns zero rows	NOT IN subquery encountered a NULL value (Section 12.2)	Use NOT EXISTS instead, or explicitly filter NULLs from the subquery
Query results seem "multiplied"/duplicated	An accidental CROSS JOIN from a missing/incorrect JOIN condition (Section 10.3)	Always double-check every JOIN has a correct, complete ON condition
Slow query despite an index existing	A function or type mismatch on the indexed column prevents index usage (Section 30.2)	Rewrite the predicate to avoid wrapping the column in a function; match data types exactly
Deadlocks under concurrent load	Transactions acquiring locks on the same rows in different orders	Enforce a consistent lock acquisition order across all transactions (Section 27.5)
GROUP BY error: "column must appear in GROUP BY"	A non-aggregated column in SELECT that isn't in the GROUP BY clause	Add the column to GROUP BY, or wrap it in an aggregate function if that's really the intent
Pagination becomes slower on later pages	Large OFFSET values forcing the database to scan and discard many rows	Switch to keyset pagination (WHERE id > :lastId) for deep pagination (Section 7.2)
Data inconsistency after a "seemingly successful" batch update	No transaction wrapping multiple related statements — a partial failure left data half-updated	Wrap related statements in an explicit transaction with COMMIT/ROLLBACK (Section 26)
Materialized view showing stale data	REFRESH MATERIALIZED VIEW not scheduled/run recently	Schedule a regular refresh job (often via Spring Batch, Section 37+), or switch to a regular View if freshness matters more than speed
Composite index not being used by a query	The query filters only on a non-leading column of the composite index (leftmost prefix rule violation, Section 23.3)	Reorder the index columns to match the actual query pattern, or add a second index for that specific filter
Application intermittently fails with "could not serialize access" errors	SERIALIZABLE isolation causing transaction conflicts/retries under concurrent load	Add application-level retry logic for serialization failures, or reconsider whether SERIALIZABLE is truly needed for that operation

36. SQL Interview Questions & Answers (Most Important Topics)

Q1. What is the difference between DDL, DML, DCL, and TCL?

DDL (CREATE, ALTER, DROP) defines database structure. DML (SELECT, INSERT, UPDATE, DELETE) manipulates data. DCL (GRANT, REVOKE) controls access permissions. TCL (COMMIT, ROLLBACK, SAVEPOINT) controls transaction boundaries.

Q2. Explain the difference between WHERE and HAVING.

WHERE filters individual rows BEFORE grouping/aggregation happens. HAVING filters the resulting GROUPS after aggregation — you can't use an aggregate function like SUM() in a WHERE clause because grouping hasn't occurred yet at that stage of query execution.

Q3. Explain the differences between INNER JOIN, LEFT JOIN, and FULL OUTER JOIN with examples.

INNER JOIN returns only rows matching in both tables. LEFT JOIN returns ALL rows from the left table, with NULLs for unmatched right-table columns. FULL OUTER JOIN returns all rows from both tables, matched where possible, NULL on whichever side lacks a match.

Real scenario: finding customers with no accounts uses LEFT JOIN + WHERE account.id IS NULL — an "anti-join" pattern.

Q4. What's the difference between EXISTS and IN, and why might EXISTS be safer?

Functionally similar for simple cases, but IN with NOT IN has a dangerous NULL trap — if the subquery returns even one NULL, the entire outer NOT IN query silently returns zero rows. NOT EXISTS is immune to this and is generally preferred for that reason, plus it often performs better since it short-circuits on the first match.

Q5. What is a correlated subquery, and how is it different from a regular subquery?

A correlated subquery references a column from the outer query, effectively re-executing once per outer row, whereas a regular subquery runs once independently. Correlated subqueries can be slower on large tables and are often rewritable as a JOIN or window function for better performance.

Q6. Explain ROW_NUMBER, RANK, and DENSE_RANK with an example involving ties.

All three assign a rank based on ORDER BY within a window. ROW_NUMBER always gives unique sequential numbers even for ties. RANK gives the same rank to ties but skips subsequent numbers (1,2,2,4). DENSE_RANK gives the same rank to ties without skipping (1,2,2,3).

Real scenario: "top N per group" (e.g., each customer's highest-balance account) is typically solved with ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY balance DESC) = 1.

Q7. What are LEAD and LAG used for?

LAG looks at a previous row's value, LEAD looks at a future row's value, both within a PARTITION BY/ORDER BY window — commonly used to compare a transaction to the previous one (e.g., detecting a sudden spike), or calculate the time gap until the next event.

Q8. Difference between UNION and UNION ALL, and which should you default to?

UNION removes duplicate rows across both result sets (implicit de-duplication, has a real performance cost). UNION ALL keeps all rows including duplicates and is faster. Default to UNION ALL unless you specifically need duplicates removed.

Q9. What is the difference between a clustered and non-clustered index?

A clustered index physically orders the table's actual data rows according to the index — only one per table. A non-clustered index is a separate structure with pointers back to the actual rows — many are allowed per table.

Q10. Explain the "leftmost prefix" rule for composite indexes.

A composite index on (A, B) can efficiently serve queries filtering on A alone, or A and B together, but NOT queries filtering on B alone — the index columns must be used starting from the leftmost column for the index to help.

Q11. What is the difference between a View and a Materialized View?

A View is a stored query, re-executed live every time it's queried — always current but only as fast as the underlying query. A Materialized View physically stores the result set, making it much faster to query but potentially stale until explicitly refreshed.

Q12. Explain ACID properties.

Atomicity (all statements in a transaction succeed or none do), Consistency (a transaction always leaves the database in a valid state honoring all constraints), Isolation (concurrent transactions don't interfere with each other's intermediate state, per the configured isolation level), Durability (once committed, data survives even a crash).

Q13. What is a deadlock, and how do you prevent it in real applications?

A deadlock occurs when two transactions each hold a lock the other needs, creating a circular wait that never resolves on its own until the database forcibly kills one transaction. Prevented by always acquiring locks on shared resources in the same, consistent order across all transactions.

Q14. Explain the difference between dirty read, non-repeatable read, and phantom read.

Dirty read: reading another transaction's uncommitted changes. Non-repeatable read: re-reading the same row within a transaction gives a different value because another transaction committed a change in between. Phantom read: re-running the same query gives a different NUMBER of rows because another transaction inserted/deleted a matching row in between.

Q15. What isolation level would you use for a critical financial reconciliation query, and why?

SERIALIZABLE — it's the only level preventing all three read phenomena (dirty, non-repeatable, and phantom reads), guaranteeing the aggregation reflects a single, consistent snapshot even under heavy concurrent activity, accepting the performance/locking cost for correctness in that specific critical, infrequent operation.

Q16. How do you diagnose and fix a slow SQL query in production?

Identify the slow query via slow query logs or APM traces, run EXPLAIN ANALYZE to see the actual execution plan, look for full table scans or unexpectedly high actual row counts, check for missing indexes or SQL anti-patterns (functions on indexed columns, leading wildcards, type mismatches) blocking index usage, then add/adjust indexes and re-verify the plan improved.

Real scenario: a UPPER(status) filter prevented index usage entirely — rewriting the predicate or adding a functional index restored fast performance.

Q17. What is a CTE, and what's the difference between a regular CTE and a Recursive CTE?

A CTE (WITH clause) is a named, temporary result set scoped to one query, improving readability by breaking complex queries into logical named steps. A Recursive CTE references itself repeatedly to traverse hierarchical/tree-structured data (like an org chart or category tree), which a non-recursive query cannot express.

Q18. What does the MERGE statement do, and why is it useful?

MERGE (or UPSERT) inserts a row if it doesn't exist or updates it if it does, in a single atomic statement — avoiding a separate check-then-insert-or-update round trip and the race condition risk that comes with doing it as two separate steps under concurrent access.

Q19. When would you put logic in a database Trigger/Stored Procedure versus application code?

Application code is preferred for most business logic — easier to test, version control, and keep portable. Triggers/procedures are reserved for concerns that must be absolutely guaranteed regardless of how data is modified (e.g., an audit trail that must fire even if someone runs raw SQL directly, bypassing the application entirely).

Q20. Describe a real SQL performance issue you've diagnosed and fixed end to end.

A search feature timed out under real production volume because the query wrapped the search column in UPPER() and used a leading wildcard LIKE pattern, both of which defeated standard indexing. Diagnosed via EXPLAIN ANALYZE showing a full sequential scan, then fixed by introducing a trigram index (pg_trgm) specifically designed for flexible text search, restoring sub-100ms performance under real data volume.

37. What Is Batch Processing and Why We Need It (Real-Time)

37.1 The Problem Batch Processing Solves

Not every operation should happen synchronously in response to a user's HTTP request — processing millions of records (calculating interest for every account, generating monthly statements, reconciling transactions with a payment gateway) would take far too long and would time out a real-time API call. Batch processing handles large volumes of data in scheduled, automated, resilient bulk operations, decoupled from any single user request.

37.2 Real Banking Examples of Batch Jobs

Batch Job	Why It's a Batch, Not a Live API Call
Nightly interest calculation across all accounts	Touches millions of rows, must run consistently and only once per day, no user waits for it live
Monthly account statement generation (PDF/email)	Bulk generation for every customer, scheduled at a specific time, not user-triggered per request
End-of-day transaction reconciliation with a payment network	Compares millions of transactions against an external file, needs error handling/retry at scale
Loan repayment due-date reminders	Scans the entire loan table for due dates, sends bulk notifications
Data migration/cleanup jobs	One-time or periodic bulk transformation of large datasets

37.3 Why Not Just Write a Simple Loop in Java?

A naive `for` loop over millions of database rows in a single method has real problems: no restart capability if it crashes at row 800,000 (must start over from row 1), no built-in retry/skip logic for individual bad records, no chunked/transactional processing (all-or-nothing on a huge dataset is fragile), and no standardized way to monitor progress or scale via parallelism. Spring Batch was built specifically to solve all of these production-grade concerns.

37.4 Batch Processing vs Real-Time Processing vs Stream Processing

Style	Latency	Real Example
Real-Time (Synchronous API)	Milliseconds — user is actively waiting	GET /accounts/1001 balance check
Batch Processing	Minutes to hours, scheduled — no one is actively waiting	Nightly interest calculation across all accounts
Stream Processing	Near-real-time, continuous, event-by-event	Kafka consumer reacting to each transaction event as it happens

38. Introduction to Spring Batch

38.1 What Spring Batch Is

Spring Batch is a lightweight, comprehensive framework for building robust, production-grade batch applications — providing reusable building blocks for reading, processing, and writing large volumes of data, with built-in transaction management, restart/retry/skip logic, and job monitoring, all as a natural extension of the Spring ecosystem.

38.2 Real Dependency

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-batch</artifactId>
</dependency>
```

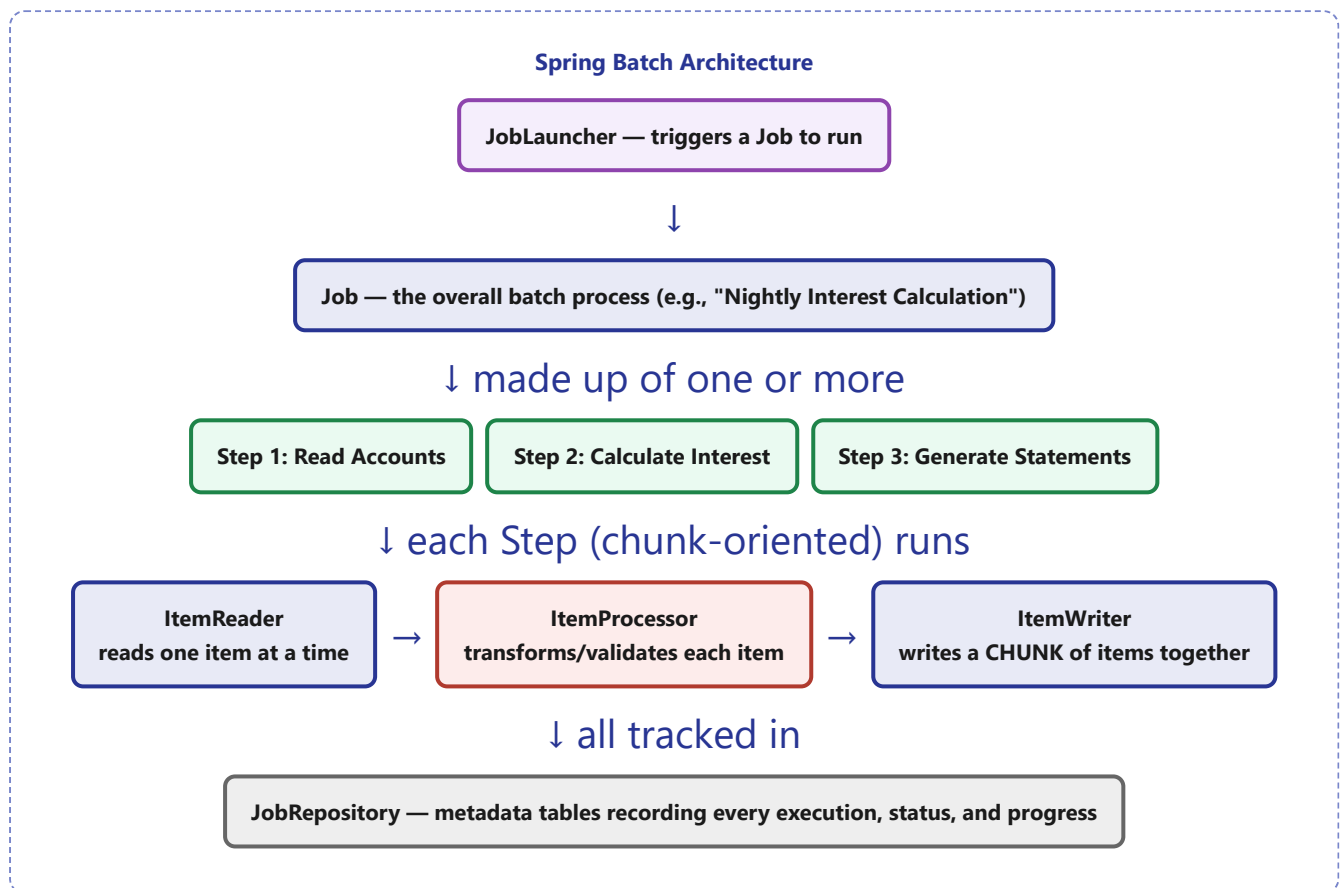
38.3 What Spring Batch Gives You Out of the Box

- A metadata schema (job_execution, step_execution tables) tracking every job run, its status, start/end time, and exact progress.
- Automatic restart from the last successful point after a failure — never reprocess already-completed chunks.
- Chunk-oriented processing with automatic transaction boundaries per chunk.
- Built-in retry and skip logic for individual problematic records, without failing the entire job.
- Listeners/hooks at every stage (job, step, chunk, item) for logging, metrics, and custom logic.
- Scaling options — parallel steps, multi-threaded steps, partitioning across multiple workers (Section 42).

Interview one-liner: "Spring Batch is to batch processing what Spring Data JPA is to the data access layer — it provides a robust, standardized framework so teams don't repeatedly hand-roll fragile, non-restartable, non-monitored batch loops themselves."

39. Spring Batch Architecture

39.1 The Full Architecture Diagram



39.2 The JobRepository — Why It's Central to Everything

The JobRepository persists metadata about every job run to a database (job_instance, job_execution, step_execution, execution_context tables) — this is precisely what enables Spring Batch's restart capability: if a job fails at chunk 4,000 of 10,000, restarting it picks up exactly where it left off, using this stored progress information, instead of reprocessing everything from scratch.

40. Spring Batch Core Concepts — Job, Step, ItemReader, ItemProcessor, ItemWriter

40.1 Job — The Overall Batch Process

```
@Configuration
public class InterestCalculationJobConfig {

    @Bean
    public Job interestCalculationJob(JobRepository jobRepository, Step calculateInterestStep) {
        return new JobBuilder("interestCalculationJob", jobRepository)
            .start(calculateInterestStep)
            .build();
    }
}
```

40.2 Step — One Logical Phase Within a Job

```
@Bean
public Step calculateInterestStep(JobRepository jobRepository,
                                PlatformTransactionManager transactionManager,
                                ItemReader<Account> reader,
                                ItemProcessor<Account, Account> processor,
                                ItemWriter<Account> writer) {
    return new StepBuilder("calculateInterestStep", jobRepository)
        .<Account, Account>chunk(500, transactionManager) // process 500 records, then commit as one
        transaction
        .reader(reader)
        .processor(processor)
        .writer(writer)
        .build();
}
```

40.3 ItemReader — Reading Data From a Source

```
@Bean
public JdbcCursorItemReader<Account> accountReader(DataSource dataSource) {
    return new JdbcCursorItemReaderBuilder<Account>()
        .name("accountReader")
        .dataSource(dataSource)
        .sql("SELECT id, balance, interest_rate FROM account WHERE status = 'ACTIVE'")
        .rowMapper((rs, rowNum) -> new Account(rs.getLong("id"), rs.getBigDecimal("balance"),
        rs.getBigDecimal("interest_rate")))
        .build();
}

// Other real reader types: FlatFileItemReader (CSV files), JpaPagingItemReader, StaxEventItemReader (XML)
```

40.4 ItemProcessor — Transforming/Validating Each Item

```
@Bean
public ItemProcessor<Account, Account> interestProcessor() {
    return account -> {
        BigDecimal interest = account.getBalance()
            .multiply(account.getInterestRate())
            .divide(BigDecimal.valueOf(100));
        account.setBalance(account.getBalance().add(interest));

        if (account.getBalance().compareTo(BigDecimal.ZERO) < 0) {
            return null; // returning null FILTERS this item out – it's simply skipped, not written
        }
        return account;
    };
}
```

40.5 ItemWriter — Writing a Chunk of Processed Items

```
@Bean
public JdbcBatchItemWriter<Account> accountWriter(DataSource dataSource) {
    return new JdbcBatchItemWriterBuilder<Account>()
        .dataSource(dataSource)
        .sql("UPDATE account SET balance = :balance WHERE id = :id")
        .beanMapped()
        .build();
    // writes the ENTIRE chunk (500 items) in one efficient batch SQL operation, then commits
}
```

Interview tip: The reader/processor work ONE ITEM AT A TIME, but the writer receives and writes an ENTIRE CHUNK together — this chunk-oriented design is exactly what enables efficient JDBC batching and clean transaction boundaries per chunk, rather than one transaction per individual row.

41. Spring Batch — Real Implementation Example (End to End)

41.1 Complete Real Job — Monthly Statement Generation

```

@Configuration
@RequiredArgsConstructor
public class StatementGenerationJobConfig {
    private final JobRepository jobRepository;
    private final PlatformTransactionManager transactionManager;
    private final DataSource dataSource;

    @Bean
    public Job statementGenerationJob(Step readAccountsStep, Step generateStatementsStep) {
        return new JobBuilder("statementGenerationJob", jobRepository)
            .start(readAccountsStep)
            .next(generateStatementsStep) // Steps run in sequence – Step 2 only starts after Step 1 fully
            .build();
    }

    @Bean
    public Step generateStatementsStep(ItemReader<Account> reader,
                                      ItemProcessor<Account, Statement> processor,
                                      ItemWriter<Statement> writer) {
        return new StepBuilder("generateStatementsStep", jobRepository)
            .<Account, Statement>chunk(200, transactionManager)
            .reader(reader)
            .processor(processor)
            .writer(writer)
            .faultTolerant()
            .skip(StatementGenerationException.class)
            .skipLimit(50) // tolerate up to 50 bad records before failing the whole step
            .retry(TransientDataAccessException.class)
            .retryLimit(3)
            .listener(new StatementStepListener())
            .build();
    }
}

@Component
public class StatementStepListener implements StepExecutionListener {
    @Override
    public ExitStatus afterStep(StepExecution stepExecution) {
        log.info("Step {} completed: read={}, written={}, skipped={}",
            stepExecution.getStepName(), stepExecution.getReadCount(),
            stepExecution.getWriteCount(), stepExecution.getSkipCount());
        return stepExecution.getExitStatus();
    }
}

```

41.2 Triggering the Job (Real Scheduled Trigger)

```
@Component
@RequiredArgsConstructor
public class MonthlyStatementScheduler {
    private final JobLauncher jobLauncher;
    private final Job statementGenerationJob;

    @Scheduled(cron = "0 0 2 1 * *") // 2 AM on the 1st of every month
    public void runMonthlyStatements() throws Exception {
        JobParameters params = new JobParametersBuilder()
            .addLocalDateTime("runDateTime", LocalDateTime.now()) // unique params ensure each run is a NEW
            .toJobParameters();
        jobLauncher.run(statementGenerationJob, params);
    }
}
```

Real scenario: "Our monthly statement job processes 2 million accounts in chunks of 200, with fault tolerance skipping up to 50 individually bad records (e.g., a customer with a malformed email preventing PDF delivery) without failing the entire multi-hour job — those 50 skipped records are logged and handled manually the next morning, while the other 1,999,950 statements go out successfully overnight."

42. Chunk-Oriented vs Tasklet Processing

42.1 Chunk-Oriented Processing (Recap)

Chunk-oriented processing (Reader → Processor → Writer, in chunks) is the standard model for "read many records, transform each, write in batches" style work — the vast majority of real batch jobs use this model.

42.2 Tasklet — For Simple, Single-Operation Steps

A Tasklet is a simpler alternative for a Step that doesn't fit the read-process-write pattern — a single method executed once (or in a loop) per Step, useful for simple operations like "delete old temp files," "call a single stored procedure," or "send one summary notification email."

```
@Bean
public Step cleanupTempFilesStep(JobRepository jobRepository, PlatformTransactionManager transactionManager) {
    return new StepBuilder("cleanupTempFilesStep", jobRepository)
        .tasklet((contribution, chunkContext) -> {
            Files.deleteIfExists(Path.of("/tmp/batch-staging-file.csv"));
            return RepeatStatus.FINISHED; // tells Spring Batch this Tasklet's work is done
        }, transactionManager)
        .build();
}
```

42.3 Chunk vs Tasklet — When to Use Which

Aspect	Chunk-Oriented	Tasklet
Best fit	Reading/transforming/writing many records	A single simple operation, or custom control flow not fitting read-process-write
Built-in batching/commit intervals	Yes — automatic, based on chunk size	No — you control the transaction boundary and logic yourself
Real example	Calculating interest for 2 million accounts	Cleaning up a staging file, or calling one stored procedure once

43. Scaling Spring Batch — Partitioning, Multi-Threading & Parallel Steps

43.1 Why Scaling Matters for Batch Jobs

A single-threaded job processing 10 million rows sequentially might take hours — real organizations need these jobs to complete within a defined batch window (e.g., overnight before markets reopen), which often requires parallelizing the work across multiple threads or even multiple machines.

43.2 Multi-Threaded Step (Simplest Scaling Option)

```
@Bean
public Step calculateInterestStep(...) {
    return new StepBuilder("calculateInterestStep", jobRepository)
        .<Account, Account>chunk(500, transactionManager)
        .reader(reader)
        .processor(processor)
        .writer(writer)
        .taskExecutor(new SimpleAsyncTaskExecutor("batch-thread")) // processes multiple chunks concurrently
        within one Step
        .throttleLimit(8) // max 8 concurrent chunk-processing
        threads
        .build();
}
```

Real gotcha: Multi-threaded steps require a thread-safe `ItemReader` (most standard readers like `JdbcPagingItemReader` are, but `JdbcCursorItemReader` is NOT) — always verify reader thread-safety before enabling this, or use partitioning instead (Section 43.3).

43.3 Partitioning — Splitting Work Across Multiple Independent Workers

Partitioned Step — Splitting a Range of Accounts

Partitioner: splits account IDs into 4 ranges



Worker 1: IDs 1-250,000

Worker 2: IDs 250,001-500,000

Worker 3: IDs 500,001-750,000

Worker 4: IDs 750,001-1,000,000

```

@Bean
public Step partitionedInterestStep(Step calculateInterestStep, Partitioner accountRangePartitioner) {
    return new StepBuilder("partitionedInterestStep", jobRepository)
        .partitioner("calculateInterestStep", accountRangePartitioner)
        .step(calculateInterestStep)
        .gridSize(4) // split the work into 4 independent partitions
        .taskExecutor(new SimpleAsyncTaskExecutor())
        .build();
}

public class AccountRangePartitioner implements Partitioner {
    @Override
    public Map<String, ExecutionContext> partition(int gridSize) {
        Map<String, ExecutionContext> partitions = new HashMap<>();
        long rangeSize = 1_000_000 / gridSize;
        for (int i = 0; i < gridSize; i++) {
            ExecutionContext context = new ExecutionContext();
            context.putLong("minId", i * rangeSize + 1);
            context.putLong("maxId", (i + 1) * rangeSize);
            partitions.put("partition" + i, context);
        }
        return partitions;
    }
}

```

Real scenario: "Our nightly interest calculation job partitions 5 million accounts into 10 parallel workers, each processing its own ID range independently and concurrently — this cut total job runtime from 3.5 hours down to about 25 minutes, comfortably fitting our overnight batch window."

43.4 Parallel Steps (When Steps Are Truly Independent)

```

@Bean
public Job parallelJob(Step generateStatementsStep, Step sendComplianceReportStep) {
    Flow statementFlow = new FlowBuilder<Flow>("statementFlow").start(generateStatementsStep).build();
    Flow complianceFlow = new FlowBuilder<Flow>("complianceFlow").start(sendComplianceReportStep).build();

    return new JobBuilder("parallelJob", jobRepository)
        .start(new FlowBuilder<Flow>("splitFlow").split(new SimpleAsyncTaskExecutor())
            .add(statementFlow, complianceFlow) // these two ENTIRELY UNRELATED steps run simultaneously
            .build())
        .end()
        .build();
}

```

44. Scheduling, Restart, Retry & Skip Logic

44.1 Scheduling — Real Options

Option	Real Use
Spring's @Scheduled with cron expressions	Simple, in-application scheduling for single-instance batch triggers
Azure DevOps / Kubernetes CronJob	Infrastructure-level scheduling, decoupled from the application's own uptime
Dedicated schedulers (Quartz, Control-M)	Enterprise-grade scheduling with complex dependency chains across many jobs/systems

44.2 Restart — Why It Works Automatically

Because the JobRepository (Section 39.2) tracks exactly which chunks completed successfully, re-launching a FAILED job with the SAME JobParameters automatically resumes from the last successfully committed chunk — completed work is never redone.

```
@Scheduled(cron = "0 0 2 * * *")
public void runNightlyJob() throws Exception {
    JobParameters params = new JobParametersBuilder()
        .addString("runDate", LocalDate.now().toString()) // SAME params every day for the SAME logical run -
        // enables restart if it failed earlier today
        .toJobParameters();
    jobLauncher.run(nightlyInterestJob, params);
}
```

Real gotcha: If a job completes SUCCESSFULLY, Spring Batch will REFUSE to run it again with the exact same JobParameters (a `JobInstanceAlreadyCompleteException`) — this is intentional, preventing accidental double-processing. Genuinely wanting to re-run a completed job (e.g., for testing) requires adding a unique parameter, like a timestamp.

44.3 Retry — Handling Transient Failures

```
.faultTolerant()
.retry(TransientDataAccessException.class) // e.g., a brief database connection blip
.retryLimit(3)
```

Retry re-attempts an individual FAILED ITEM a limited number of times before giving up on it — appropriate for genuinely transient issues (a momentary network/database hiccup), not for permanent data problems (a malformed record will fail identically every retry).

44.4 Skip — Tolerating a Limited Number of Bad Records

```
.faultTolerant()
.skip(DataValidationException.class)    // e.g., a specific customer record has invalid/missing data
.skipLimit(100)                        // tolerate up to 100 total skips before failing the whole step
.noSkip(FileNotFoundException.class)    // but NEVER skip this – a missing input file is a fatal condition
```

Real scenario: "Our monthly statement job skips (rather than fails entirely) any individual customer record with a malformed email address, up to a limit of 100 skips — those specific accounts are logged to a follow-up table for manual review the next morning, while the job completes successfully for the other 99.995% of customers instead of the whole multi-hour run failing over a handful of bad records."

45. Spring Batch Real-Time Scenarios & Issues

Issue	Cause	Real Solution
Job runs but reprocesses everything from scratch after a crash	Different JobParameters used on retry, creating a NEW JobInstance instead of resuming the failed one	Use the SAME JobParameters (e.g., same runDate string) when relaunching after a genuine failure
"JobInstanceAlreadyCompleteException" when re-running for testing	Spring Batch refuses to rerun a job that already completed successfully with identical parameters	Add a unique parameter (timestamp) when deliberately re-running a completed job for testing purposes
Multi-threaded step produces corrupted/duplicate data	A non-thread-safe ItemReader used with a taskExecutor	Use a thread-safe reader (JdbcPagingItemReader) or switch to partitioning instead
One bad record fails the entire multi-hour job	No fault tolerance (skip/retry) configured	Add .faultTolerant().skip(...).skipLimit(...) for genuinely tolerable, isolated bad records
Batch job runs past its allotted overnight window	Single-threaded processing of a very large dataset	Partition the work across multiple parallel workers (Section 43.3)
Database connection pool exhausted during a large batch run	Batch job's connection pool conflicting with the application's regular connection pool	Use a dedicated, appropriately-sized connection pool/datasource specifically for batch processing
Retry keeps failing on a permanently bad record	Retry configured for a non-transient error type (e.g., a data validation failure, not a network blip)	Use skip (not retry) for genuinely invalid data; reserve retry strictly for transient infrastructure issues
Job metadata tables growing unbounded over time	No cleanup/archival policy for old job_execution/step_execution history	Periodically archive or purge old batch metadata according to a defined retention policy

46. Spring Batch Interview Questions & Answers

Q1. What is Spring Batch, and why not just write a plain Java loop for bulk processing?

Spring Batch is a framework for robust, production-grade batch processing, providing restart capability, chunk-based transaction management, retry/skip logic, and monitoring out of the box — a plain loop lacks all of this, meaning a crash partway through requires reprocessing everything from scratch, with no standardized error handling at scale.

Q2. Explain the relationship between Job, Step, and chunk-oriented processing.

A Job is the overall batch process, made up of one or more Steps executed in sequence (or in parallel). A chunk-oriented Step reads items one at a time via an `ItemReader`, transforms each via an `ItemProcessor`, and writes an entire chunk together via an `ItemWriter`, committing one transaction per chunk.

Q3. How does Spring Batch enable restart after a failure?

The `JobRepository` persists metadata about every job/step execution, including exactly which chunks completed successfully. Relaunching a failed job with the SAME `JobParameters` resumes from the last successfully committed chunk instead of reprocessing already-completed work.

Q4. What's the difference between chunk-oriented processing and a Tasklet?

Chunk-oriented processing is for reading/transforming/writing many records via the Reader-Processor-Writer pattern. A Tasklet is a simpler, single-method Step for operations that don't fit that pattern, like cleaning up a file or calling one stored procedure.

Q5. What is the difference between retry and skip in Spring Batch?

Retry re-attempts a failed item a limited number of times, appropriate for transient issues like a brief database connection blip. Skip tolerates a limited number of permanently bad items by excluding them from processing (up to a configured `skipLimit`) so the overall job can still succeed despite a handful of genuinely invalid records.

Q6. How do you scale a Spring Batch job to process millions of records faster?

Options include multi-threaded steps (processing multiple chunks concurrently within one step, requiring a thread-safe reader), partitioning (splitting the data into independent ranges processed in parallel by separate worker steps), and running genuinely independent steps in parallel via a split flow.

Real scenario: partitioning 5 million accounts into 10 parallel workers cut a nightly interest calculation job from 3.5 hours to about 25 minutes.

Q7. Why would a multi-threaded step corrupt data, and how do you avoid it?

Some `ItemReaders` (like `JdbcCursorItemReader`) aren't thread-safe — using them with a `taskExecutor` across multiple threads can cause corrupted or duplicated reads. The fix is using a thread-safe reader (like `JdbcPagingItemReader`) or switching to partitioning, which gives each worker its own independent reader instance entirely.

Q8. What happens if you try to rerun a Spring Batch job that already completed successfully with the same parameters?

Spring Batch throws a `JobInstanceAlreadyCompleteException` — this is intentional, preventing accidental double-processing of the same logical batch run. To deliberately rerun (e.g., for testing), you must supply a new/unique `JobParameter`.

Q9. How would you design a batch job to process 10 million records without failing entirely due to a handful of bad records?

Configure fault tolerance with `.faultTolerant().skip(SpecificException.class).skipLimit(N)` — this tolerates up to N individually bad records (logging them for follow-up) while letting the job successfully complete for all other valid records, rather than failing the entire multi-hour run over a small number of edge cases.

Q10. How does Spring Batch fit into a real organization's overall architecture alongside microservices, APIs, and databases?

Spring Batch typically runs as its own scheduled job (or a dedicated batch-processing service) reading from and writing to the same core databases used by the real-time microservices, often triggered by Kubernetes CronJobs or an enterprise scheduler, and monitored via the same Actuator/Prometheus/Grafana observability stack used for the rest of the platform — handling the large-volume, non-interactive workloads that real-time APIs are not designed for.

47. Final Summary & Cheat Sheet

DBMS foundations: flat files lacked integrity/concurrency/query power — a DBMS solves this; RDBMS suits strongly relational, transactional data (banking cores), NoSQL suits flexible-schema/high-write-scale needs (catalogs, logs) — real organizations use both together (polyglot persistence).

SQL fundamentals: DDL/DML/DCL/TCL categorize every SQL command; SELECT/WHERE/GROUP BY/HAVING follow a strict logical execution order distinct from written order.

Joins & subqueries: INNER for matches-only, LEFT for "everything plus matches," NOT EXISTS is safer than NOT IN due to the NULL trap.

Window functions (ROW_NUMBER, RANK, LAG/LEAD, PARTITION BY) solve "top N per group," deduplication, and row-to-row comparisons elegantly, without collapsing detail like GROUP BY does.

Indexes: the single biggest real-world performance lever — understand the leftmost-prefix rule, and always verify actual index usage via EXPLAIN ANALYZE rather than assuming.

Transactions: ACID guarantees make relational databases safe for financial systems; choose isolation levels deliberately, and prevent deadlocks via consistent lock ordering.

Advanced SQL: CTEs (including recursive) improve readability and enable hierarchical queries; MERGE/UPSERT avoids race conditions in insert-or-update logic; keep business logic in application code by default, reserving triggers/procedures for guaranteed, unbypassable enforcement.

Spring Batch: the production-grade framework for large-volume, scheduled, restartable, fault-tolerant batch processing — chunk-oriented Job/Step/Reader/Processor/Writer architecture, with partitioning and multi-threading for real scale, and skip/retry for graceful handling of imperfect real-world data.

47.1 Quick Cheat Sheet

Term	One-Line Meaning
RDBMS vs NoSQL	Strict relational/ACID vs flexible-schema/horizontally-scalable
DDL / DML / DCL / TCL	Structure / Data / Access / Transaction command categories
WHERE vs HAVING	Filters rows before grouping vs filters groups after aggregation
NOT EXISTS vs NOT IN	NOT EXISTS is immune to the NULL trap that silently breaks NOT IN
ROW_NUMBER / RANK / DENSE_RANK	Unique sequential / ties-skip-numbers / ties-no-skip ranking
Composite Index Leftmost Rule	Only usable starting from its leftmost column(s)
ACID	Atomicity, Consistency, Isolation, Durability
Deadlock Fix	Always acquire locks in a consistent, agreed order
Execution Plan	Shows exactly how a query will run — the primary SQL tuning tool
CTE / Recursive CTE	Named temporary result set / self-referencing for hierarchical data
MERGE / UPSERT	Insert-or-update in one atomic, race-condition-free statement
Spring Batch Job/Step	Overall process / one logical phase within it

Chunk-Oriented Processing	Read-Process one at a time, Write an entire chunk together
Partitioning	Splits a large dataset across independent parallel workers
Skip vs Retry	Tolerate permanently bad records vs re-attempt transient failures

Use this guide alongside the JDBC/Hibernate/Spring Data JPA guide, the Spring & Spring Boot guide, the Microservices guide, and the Observability & Monitoring guide — together they cover the complete real-world Java full-stack developer journey from raw SQL and database fundamentals, through the ORM/data-access layer, to building, scaling, and batch-processing production-grade Spring Boot systems.